

**UNIVERSITE DE TECHNOLOGIES ET DE MANAGEMENT
UTM**

Filière : 1^{ère} année Technologique

**SUPPORT DE COURS
UE : Electronique numérique et systèmes
programmés 1 (L1)**

Enseignant : COULIBALY Souleymane, tel : 70 75 88 39 ou 68 76 42 77

ANNEE : 20....-20....

ELECTRONIQUE NUMERIQUE 1

OBJECTIFS

- Comprendre les différents systèmes de codage couramment utilisés dans les systèmes informatiques
- Encoder et décoder des informations (conversion de données)
- Concevoir un code pondéré
- Effectuer des opérations arithmétiques et logiques.
- Concevoir un circuit combinatoire (logigramme)
- Comprendre les principes fondamentaux menant à la conception des systèmes numériques
- Analyser, concevoir et simuler des circuits logiques de complexité moyenne
- Réaliser l'importance des notions relatives aux circuits logiques dans le domaine des technologies de l'information (TI) et de les appliquer dans d'autres domaines

I. Introduction aux systèmes numériques

Systèmes analogique et numérique

1. Introduction : De l'analogique au numérique

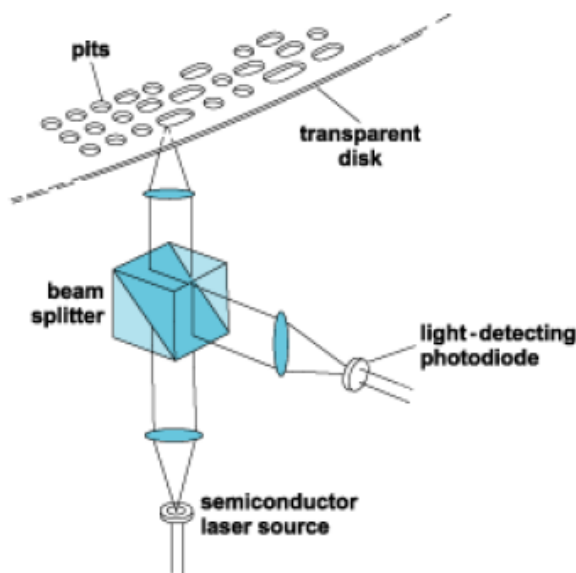
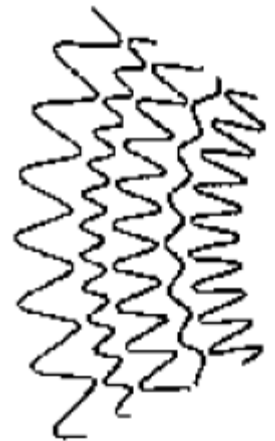
Entre un disque « noir » et un disque « compact » il y a une différence de principe :

Le premier est *analogique*, le second *numérique*. Que signifient ces termes ?

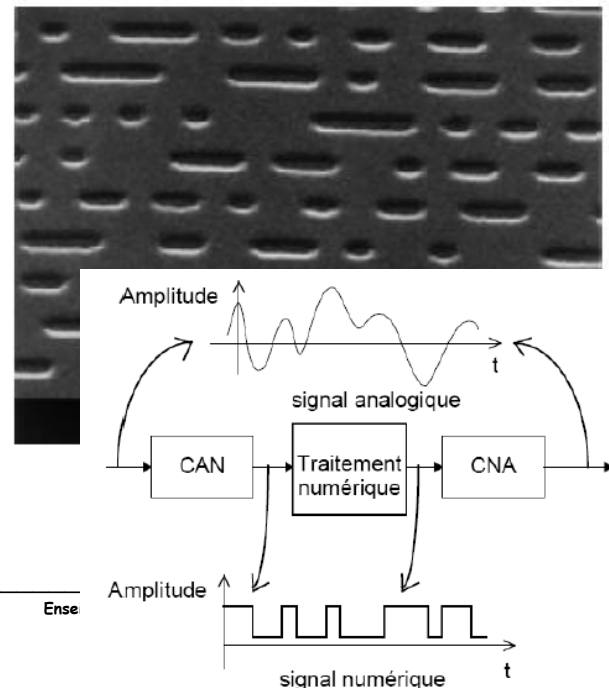
- **Le mot *analogique* évoque ressemblance**, si on regarde avec un microscope une partie de sillon d'un disque noir on verra une sorte de vallée sinueuse dont les flancs reproduisent, à peu près, la forme des signaux électriques transmis aux hautparleurs.

A un son grave correspondent des sinuosités qui ont une période spatiale grande (quelques mm pour 100 Hz), à un son aigu correspondent des sinuosités dont la période spatiale est plus petite (quelques centièmes de mm pour 10 kHz). De même, l'amplitude des sinuosités reproduit, grosso-modo, l'amplitude du son que l'on souhaite reproduire.

- **Le mot *numérique* évoque nombre**. Si on regarde au microscope (Grossissement supérieur à 100) une plage d'un disque compact on verra une sorte de pointillé de trous ovales, presque identiques, répartis de façon irrégulière sur des pistes quasi-circulaires. Aucun rapport de forme entre le son enregistré et l'allure de la gravure ne peut être observé, présence ou absence de trous constituent les deux valeurs possibles d'un chiffre en base 2. Ces chiffres, regroupés par paquets de 16, constituent des nombres entiers dont la valeur est l'image, via un code, de l'amplitude du signal sonore.



Le



passage d'un monde à l'autre se fait par des *convertisseurs analogique numérique (CAN) et numérique analogique (CNA)*, dont nous n'étudierons pas ici le fonctionnement. La différence de principe évoquée plus haut se retrouve évidemment quand on observe le fonctionnement des circuits : *un circuit analogique manipule des signaux électriques qui peuvent prendre une infinité de valeurs, qui sont en général les fonctions continues du temps, un circuit numérique manipule des signaux qui ne peuvent prendre qu'un nombre fini (généralement 2) de valeurs conventionnelles, sans rapport avec le contenu de l'information, qui sont des fonctions discontinues du temps.*

2. Deux niveaux électriques (0V ; +Vcc) et états logiques (0 ;1 : le bit)

Dans un circuit électronique les signaux sont représentés par des niveaux électriques. La grandeur physique significative que l'on utilise le plus souvent est la tension (un signal électrique peut très bien être un courant), sauf précision contraire explicite la « valeur » d'un signal électrique binaire se mesurera donc en volts.

Dans un système numérique tous les potentiels sont mesurés par rapport à un potentiel de référence, la masse, qui est une équipotentielle commune à tous les circuits. Dans un système numérique les signaux numériques sont représentés par des valeurs binaires qui ne peuvent prendre que deux valeurs (0 et 1). Une variable binaire porte le nom de *bit*, contraction de binary digit, littéralement chiffre en base 2.

3. Conventions logiques

Une entrée ou une sortie d'un circuit numérique ne peut prendre que deux valeurs, notées généralement **H**, pour High (haut), et **L**, pour Low (bas) : 3 et 0,2 volts sont des valeurs typiques fréquemment rencontrées.

La valeur d'un signal représente en général quelque chose :

- chiffre en base deux, 0 ou 1,
- valeur d'une variable logique, vrai ou faux,
- état d'un opérateur, actif ou inactif,
- état d'une porte, ouverte ou fermée,
- état d'un moteur, arrêté ou en marche,
- etc.

Traditionnellement on qualifie de convention logique *positive* l'association entre **H** et 1, ou vrai, ou actif, et de convention logique *négative* l'association entre **H** et 0, ou faux, ou inactif.

4. Immunité au bruit

L'un des intérêts majeurs des signaux numériques est leur grande robustesse vis à vis des perturbations extérieures. L'exemple des enregistrements sonores en est une bonne illustration. Les disques noirs reproduisent les altérations mécaniques dues à leur manipulation et à leur vieillissement, le signal sonore s'en retrouve bruité. Le signal numérique ne vieillit pas. Seul son support est altérable.

Au niveau du bit, la protection repose sur le fait que l'information n'est pas contenue dans l'amplitude du signal. Un signal analogique direct a une forme qui est l'image de l'information à transmettre. Toute perturbation à cette forme se traduit par une déformation de l'information associée.

L'amplitude d'un signal numérique n'a pas de rapport avec l'information véhiculée, la seule contrainte est que le système soit encore capable de différencier sans ambiguïté un niveau haut et un niveau bas. L'écart entre ces deux niveaux étant grand, seule une perturbation de grande amplitude pourra provoquer une erreur de décision.

Au niveau du système

Les valeurs élémentaires (bits) sont regroupées en paquets pour former des *mots*, ces mots doivent obéir à certaines règles de construction, des *codes*. On crée des codes qui permettent de détecter et, dans une certaine mesure, de corriger des erreurs.

5. Codage de l'information et numération : Principe de codage

Toute information traitée par les systèmes informatiques devra pouvoir être représentée sous une forme binaire compréhensible par la machine, dans le cas qui nous intéresse, sous forme **binaire**. Ce passage d'une information, d'un langage compréhensible par l'homme, à un langage compréhensible par la machine s'appelle codage ou codification.

Il existe de nombreuses possibilités de codage de l'information, citons dès à présent :

Binaire, Hexadécimal, BCD, ASCII...

Rappels sur la numération en électronique numérique

En électronique numérique les nombres sont codés en binaire, deux états 0 et 1.

Généralement 0 est codé par une tension de 0v et 1 par une tension de 5v. Avec la réduction de la taille des transistors intégrés sur une puce, le 1 est aussi représenté par une tension de 3v voir 1,2v.

II. LES SYSTEMES DE NUMERATION

La numération consiste à quantifier des matériels ou des objets par des symboles. La nécessité de quantifier notamment les échanges commerciaux s'est faite dès la structure de la vie sociale. Les tentatives de représentation symbolique furent nombreuses : bâtons, chiffres romains..., avant que ne s'impose la numération arabe universellement adoptée étant donnée sa bonne capacité à traiter les calculs courants.

1.1 Principe de numération

La numération représente un nombre par la juxtaposition de symboles appelés chiffres, pris dans un ensemble.

Lorsqu'un nombre est écrit, la position respective des chiffres détermine leurs poids : ... milliers, centaines, dizaines, unités, dixièmes, centièmes....

Tout nombre N exprimé dans un système de numération de base b a pour écriture détaillée :

$$(N)_b = S(a_n b^n + a_{n-1} b^{n-1} + \dots + a_1 b^1 + a_0 b^0 + a_{-1} b^{-1} + \dots + a_{-m} b^{-m})$$

où les a_i sont les symboles nécessaires pour exprimer n'importe quel nombre dans une base donnée avec $0 \leq a_i \leq b-1$ et S le signe de N.

Ce même nombre $(N)_b = s a_n a_{n-1} \dots a_1 a_0, a_{-1} a_{-2} \dots a_{-m}$

$$\begin{aligned} \text{Exemple : } (1437.28)_{10} &= 1*1000 + 4*100 + 3*10 + 7*1 + 2*1/10 + 8*1/100 \\ &= 1*10^3 + 4*10^2 + 3*10^1 + 7*10^0 + 2*10^{-1} + 8*10^{-2} \end{aligned}$$

1.2 Les bases de numération

Le nombre d'éléments a_i ou de symboles différents dans un système de numération détermine sa base.

1.2.1 La base 10 ou système de numération décimale

C'est le système de numération utilisée dans la vie courante ; elle comprend 10 symboles différents : 0,1,2,3,4,5,6,7,8,9.

Un nombre N exprimé dans le système décimal a pour écriture explicite :

$$\begin{aligned} (1437.28)_{10} &= 1*1000 + 4*100 + 3*10 + 7*1 + 2*1/10 + 8*1/100 \\ &= 1*10^3 + 4*10^2 + 3*10^1 + 7*10^0 + 2*10^{-1} + 8*10^{-2} \end{aligned}$$

Les puissances de 10 sont appelées les poids ou les valeurs de position. Le poids est égal à la base élevée à la puissance de son rang.

1.2.2 La base 2 ou système de numération binaire

C'est la base la plus utilisée en électronique numérique. Elle dispose de 2 symboles différents : 0 et 1, appelés bits (binary digit) ou chiffre binaire ou encore élément binaire.

$$\text{Exemple : } (10110)_2 = 1*2^4 + 0*2^3 + 1*2^2 + 1*2^1 + 0*2^0.$$

C'est la base utilisée pour coder toute information de façon à permettre sa manipulation par des opérateurs ou des machines et de pouvoir la traiter ou la transmettre.

NB : Définitions

Bit : Binary digit ; élément binaire : 0 ou 1

Quartet : ensemble formé de 4 bits : 0000 ; 1010 ; 0010 ; 0111 ; ...

Octet ou Byte : ensemble formé de 8 bits : 01010100 ; 11111111 ; 00000000 ; 10001011...

Mot : ensemble quelconque de bits

1 Kilobit = 1024 bits soit 2^{10} .

1 Méga : 2^{20} .

L.S.B. : Least Significant Bit: Bit le moins significatif ou bit de poids faible (situé le plus à droite).

M.S.B. : Most Significant Bit: Bit le plus significatif ou bit de poids fort (situé le plus à gauche).

Convertir : 2 octets =bits
 1,44 Mbits =bits
 64 Koctets =bits

1.2.3 La base 8 ou système de numération octale

Cette base est plus utilisée par les informaticiens et contient 8 symboles. Les symboles sont {0,1,2,3,4,5,6,7}.

Exemple : $(426)_8$

1.2.4 La base 16 ou système de numération hexadécimale

Elle est apparue avec la logique programmée et les microprocesseurs. L'ensemble des symboles contient 16 éléments qui sont les dix premiers chiffres décimaux et les six premières lettres de l'alphabet :

Digits : {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F}.

Equivalent décimal : {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15}.

Exemple : $(4C8)_{16}$; $(EDF)_{16}$; $(3F)_{16}$; $(259)_{16}$.

1.3 Conversion d'une base dans une autre

1.3.1 Tableau de correspondance entre les bases

Décimal (base 10)	Binaire (base 2)	Octal (base 8)	Hexadécimal (base 16)
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10
17	10001	21	11
20	10100	24	14
30	11110	36	1E
40	101000	50	28
50	110010	62	32
60	111100	74	3C
70	1000110	106	46
100	1100100	144	64

1.3.2 Conversion d'un nombre écrit en base b vers un nombre écrit en base 10

On utilise la forme polynomiale par décomposition du nombre écrit en base B.

Exemple :

$$(1001)_2 = (\dots)_{10}$$

$$(1001)_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = (9)_{10}$$

$$(11000101)_2 = (\dots)_{10}$$

$$(10110,11)_2 = (?)_{10}$$

$$(10110,11)_2 = 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 + 1 \cdot 2^{-1} + 1 \cdot 2^{-2}$$

$$(53)_8 = (?)_{10}$$

$$(426)_8 = (\dots)_{10}$$

$$(51)_8 = (\dots)_{10}$$

$$(56)_{16} = (?)_{10}$$

$$(C2)_{16} = (\dots)_{10}$$

$$(BE,68)_{16} = (\dots)_{10}$$

$$(FF)_{16} = (\dots)_{10}$$

1.3.3 Conversion d'un nombre écrit en base 10 vers un nombre écrit en base b

On procède à une division successive de la partie entière du nombre par la base b en gardant les restes et une multiplication successive de la partie fractionnaire du nombre par b. Prendre les restes successifs de la fin au début pour former le nombre converti.

Exemples

$$(37,48)_{10} = (?)_2$$

$$\text{partie entière} = 37$$

$$\text{partie fractionnaire} = 0.48$$

$$(147)_{10} = (\dots)_2$$

$$(27,64)_{10} = (\dots)_2$$

$$(278)_{10} = (\dots)_8$$

$$(549)_{10} = (\dots)_8$$

$$(3807)_{10} = (\dots)_{16}$$

$$(1000)_{10} = (\dots)_{16}$$

1.3.4 Conversion d'un nombre écrit en base b vers un nombre écrit en base b^k et inversement

On regroupe les termes k par k à partir de la virgule et remplacer chaque groupe par son correspondant.

Exemples

Base 2 vers base 8 ($8 = 2^3$) : regrouper les bits en 3 par 3 en commençant par la droite pour la partie entière ou la gauche pour la partie fractionnaire et remplacer chaque groupe par son correspondant en octal.

$$\begin{aligned} N &= (1111011001,1011101)_2 \\ &= (001\ 111\ 011\ 001, 101\ 110\ 100)_2 \\ &= (1\ 7\ 3\ 1, 5\ 6\ 4)_8 \end{aligned}$$

$$(10110010)_2 = ()_8$$

$$(10,1)_2 = ()_8$$

Base 8 vers base 2

$$\begin{aligned} N &= (3\ 6)_8 \\ &= (011\ 110)_2 \end{aligned}$$

$$(64)_8 = ()_2 = \dots\dots\dots$$

$$(276,32)_8 = ()_2 = \dots\dots\dots$$

Base 2 vers base 16 ($16 = 2^4$) : regrouper les bits en 4 par 4 en commençant par la droite pour la partie entière ou la gauche pour la partie fractionnaire et remplacer chaque groupe par son correspondant en hexadécimal.

$$\begin{aligned} N &= (1111011001,1011101)_2 \\ &= (0011\ 1101\ 1001,1011\ 1010)_2 \\ &= (3\ D\ 9, B\ A)_{16} \end{aligned}$$

$$(10001)_2 = ()_{16} = \dots\dots\dots$$

$$(11101,11)_2 = ()_{16} = \dots\dots\dots$$

Base 16 vers base 2

$$N = (2\ F)_{16} = (0010\ 1111)_2$$

$$(5A)_{16} = ()_2$$

$$(3CD)_{16} = ()_2$$

$$(BE,7)_{16} = ()_2$$

1.3.5 Conversion d'un nombre écrit en base b^p vers un nombre écrit en base b^k et inversement

La conversion d'une base b^p à b^k est obtenue en passant d'abord par la base b

Exemples

$$(32)_8 = (?)_{16} \quad (8 = 2^3 \text{ et } 16 = 2^4)$$

$$(32)_8 = (011\ 010)_2$$

$$(011\ 010)_2 = (0001\ 1010)_2 = (1A)_{16}$$

$$(8C)_{16} = (?)_8$$

$$(47)_8 = ()_{16}$$

$$(AD)_{16} = ()_8$$

1.4 Arithmétique binaire

ADDITION	SOUSTRACTION	MULTIPLICATION	DIVISION
$\begin{array}{r} 1101101 \\ + \\ 1001111 \\ \hline \end{array}$	$\begin{array}{r} 110110101 \\ - \\ 1010011 \\ \hline \end{array}$	$\begin{array}{r} 10111 \\ * \\ 101 \\ \hline \end{array}$	$\begin{array}{r l} 1101101 & 1011 \\ \hline & = \end{array}$
=	=	=	

1.5 Représentation binaire des nombres signés

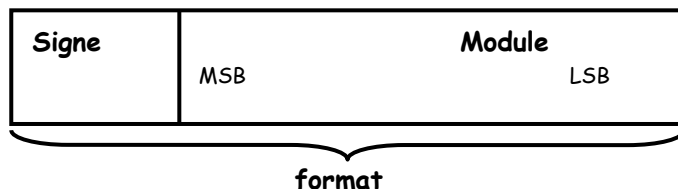
Dans les conversions vues précédemment, il n'est pas fait allusion au signe. Seul la valeur absolue des nombres peut être représentée par un ensemble d'éléments binaires. Il existe plusieurs modes de représentations des nombres signés en fonction des calculs à effectuer et des caractéristiques technologiques des systèmes utilisés. Les nombres doivent être écrits avec un format (nombre de bits) fixe. Les formats usuels sont 4 bits, 8 bits, 16 bits.

1.5.1 Représentation module + signe

Un élément binaire est ajouté au module pour la représentation du signe. C'est le bit de signe

0 → + (nombre positif)

1 → - (nombre négatif)



Exemple : sur format 8 bits

$(+23)_{10} = 0\ 0010111$

$(-23)_{10} = 1\ 0010111$

Signe Module

Si le format d'un nombre N est de n éléments binaires, ce nombre est compris entre :

$-(2^{n-1} - 1)$ et $+(2^{n-1} - 1)$

Exemple : format 4 bits ; $n - 1 = 3$

$-(2^{n-1} - 1) = -(2^3 - 1) = -7$

$+(2^{n-1} - 1) = +(2^3 - 1) = +7$

ce nombre est compris entre -7 et +7

Remarque : Il existe 2 représentations différentes du nombre 0.

Ce mode de représentation présente l'avantage d'être simple mais est peu adaptée aux calculs.

Format 4 bits

Nombre positifs			Nombre négatif		
7	0	111	-7	1	111
6	0	110	-6	1	110
5	0	101	-5	1	101
4	0	100	-4	1	100
3	0	011	-3	1	011
2	0	010	-2	1	010
1	0	001	-1	1	001
0	0	000	0	1	000

1.5.2 Représentation des nombres en complément à 2

La méthode du complément à 2 permet de représenter les nombres négatifs en gardant les méthodes de calcul habituelles du système binaire. Le complément à 2 est l'opposé arithmétique. On obtient le complément à 2 ou l'opposé arithmétique en passant par deux étapes :

1. Le complément à 1 : on inverse chaque bit (prendre son complément).
2. Le complément à 2 : on ajoute 1 au complément à 1

Ne pas oublier le bit de signe : Nombre positif ≥ 0 : bit de signe = 0

Nombre négatif < 0 : bit de signe = 1

Exemple : $A = (0111)_2 = (7)_{10}$

Le complément à 1 de $A = \bar{A} = 1000$

Le complément à 2 de $A = \bar{A} + 1 = (1001)_2 = -(7)_{10}$

De façon générale, sur des mots de n bits ou de format n bits, on pourra représenter les nombres compris entre : -2^{n-1} et $+2^{n-1} - 1$

Exemple: Format 4 Bits : $-8 \leq N \leq +7$

7	0 111
6	0 110
5	0 101
4	0 100
3	0 011
2	0 010
1	0 001
0	0 000
-1	1 111
-2	1 110
-3	1 101
-4	1 100
-5	1 011
-6	1 010
-7	1 001
-8	1 000

Remarque : le bit de signe fait partie du format. Pour connaître la valeur d'un nombre négatif, il suffit de calculer le nombre opposé.

En utilisant cette méthode de représentation des nombres négatifs, nous pouvons maintenant réaliser des opérations arithmétiques mettant en jeux des nombres négatifs.

1.5.3 Opération avec les nombres signés

a) L'addition

L'addition de nombres signés est utilisée dans la plupart des circuits numériques. Trois cas peuvent se présenter :

- *addition de deux nombres positifs,
- *addition de deux nombres de signes contraires,
- *addition de deux nombres négatifs.

La méthode consiste à remplacer les nombres négatifs par leur complément à deux avant l'addition. Soustraire un nombre B d'un autre A revient à additionner au nombre A le complément à 2 du nombre B (bit de signe compris).

Addition de 2 nombres sur format 4 bits

3	0011	4	0100	
+	+	+	+	
2	0010	5	0101	
5	0101	9	1001	← Résultat faux →dépassement

-3	1101	6	0110
+	+	+	+
-2	1110	-4	1100
-5	1101	2	10010

Retenues à négliger

Exercices :

1) Format 4 bits : $(0011)_2 + (1001)_2 =$ $(0111)_2 + (1110)_2 =$

2) Format 8 bits : $(17)_{10} + (12)_{10} = (\dots)_2$
 $(17)_{10} - (12)_{10} = (\dots)_2$
 $(-17)_{10} + (-12)_{10} = (\dots)_2$
 $(00010101)_2 + (11110010)_2 = (\dots)_2$
 $(00101101)_2 + (10001111)_2 = (\dots)_2$

3) Sur format 8 bits trouver le plus grand nombre et le plus petit.

4) Convertir en base 2 ou 16 sur format 8 bits

$(-13)_{10} = (\dots)_2$ $(-27)_{10} = (\dots)_2$
 $(-52)_{10} = (\dots)_{16}$ $(64)_{10} = (\dots)_{16}$

b) Multiplication

Si les deux nombres sont positifs : Faire la multiplication comme d'habitude.

Si les deux nombres sont négatifs : prendre leur complément à 2 et faire la multiplication.

Si l'un des nombres est négatif : prendre son complément à 2 et faire la multiplication.

c) Division

Si les deux nombres sont positifs : c'est la méthode habituelle.

Si les deux nombres sont négatifs : prendre leur complément à 2 et faire la division.

Si l'un des nombres est négatif : prendre son complément à 2 et faire la division.

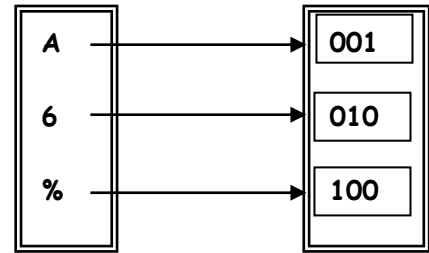
Conclusion : le complément à 2 permet de travailler avec les nombres négatifs mais il faut choisir un format fixe et négliger les retenues.

III. LES CODES

1) Notion de codage

Le codage est une opération qui établit une correspondance entre les éléments de 2 ensembles. Il consiste à faire correspondre à tout caractère (lettre, chiffre, signe de ponctuation...) un symbole ou ensemble de symboles appelé mot de code. L'opération inverse est appelée décodage.

A chaque nombre, caractère ou signe correspond une combinaison binaire appelée mot - code. L'ensemble des mots - code forme le code.



2) Les codes binaires

Ce sont des combinaisons de bits permettant d'exprimer des informations quelconques sous forme binaire.

2.1) Les codes numériques pondérés

Ce sont des codes qui donnent à chaque combinaison une équivalence décimale. Chaque rang d'élément binaire a un poids précis.

a) Le code binaire naturel

C'est la représentation octale et hexadécimale. Il permet de faire des opérations

(N) ₁₀	BINAIRE	OCTALE
0	000	0
1	001	1
2	010	2
3	011	3
4	100	4
5	101	5
6	110	6
7	111	7

(N) ₁₀	binaire	HEXADECIMAL	(N) ₁₀	binaire	HEXADECIMAL
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F

Exemple : $(0100)_2 = 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0$

Les poids utilisés sont 2^0 , 2^1 , 2^2 et 2^3 soit 1, 2, 4 et 8

b) Le code DCB (Décimal Codé Binaire) ou BCD (Binary Coded Decimal)

C'est la représentation binaire du système décimal des chiffres de 0 à 9. Chaque chiffre a son code. Il y a alors 10 mots-codes. Exemple BCD 8421

BCD 8 4 2 1	Décimal
0 0 0 0	0
0 0 0 1	1
0 0 1 0	2
0 0 1 1	3
0 1 0 0	4
0 1 0 1	5
0 1 1 0	6
0 1 1 1	7
1 0 0 0	8
1 0 0 1	9

Ainsi le nombre $(792)_{10}$ devient
 $(0111\ 1001\ 0010)_{BCD}$

c) Le code de AIKEN 1242 ou 2421

C'est un code auto complémentaire. La représentation binaire de 2 chiffres dont la somme est 9 sont complémentaires l'une de l'autre.

- les chiffres de 0 à 4 sont codés en binaire pur
- les chiffres de 5 à 9 sont codés en complémentant les bits des chiffres de 4 à 0 ou en ajoutant 6 auparavant. C'est à dire 5 aura le codage de 5+6 soit 11 et 6 aura le codage de 6+6 soit 12 etc...

Décimal	Aïken 2 4 2 1	
0	0 0 0 0	→ Codé 0
1	0 0 0 1	→ Codé 1
2	0 0 1 0	→ Codé 2
3	0 0 1 1	→ Codé 3
4	0 1 0 0	→ Codé 4
5	1 0 1 1	→ Codé 5 + 6 = 11
6	1 1 0 0	→ Codé 6 + 6 = 12
7	1 1 0 1	→ Codé 7 + 6 = 13
8	1 1 1 0	→ Codé 8 + 6 = 14
9	1 1 1 1	→ Codé 9 + 6 = 15

On remarque bien :

$$0 + 9 = 9 = 0000 + 1111 = 1111$$

$$1 + 8 = 9 = 0001 + 1110 = 1111$$

$$2 + 7 = 9 = 0010 + 1101 = 1111$$

$$3 + 6 = 9 = 0011 + 1100 = 1111$$

$$4 + 5 = 9 = 0100 + 1011 = 1111$$

Ce code est utilisé dans certains calculateurs pour effectuer les soustractions par addition de la forme complémentaire.

d) le code EXCESS 3 (XS3)

IL a été créé pour permettre la réalisation simple des opérations de soustraction.

Ce code XS3 est obtenu en ajoutant 3 auparavant au code BCD.

$$XS3 = BCD + 0011$$

2.2) Les codes numériques non pondérés

Ce sont des codes dont la constitution n'est pas liée à une notion de poids affectant chaque bit.

a) Le code de GRAY ou code binaire réfléchi

On l'appelle aussi code à distance unité. La distance entre 2 mots-code est le nombre d'éléments binaires qui le diffère.

Lorsque l'on passe d'une ligne à l'autre un seul bit change de valeur.

On l'utilise pour le codage optique de positions ou dispositif de comptage.

(N) ₁₀	code de Gray
0	000
1	001
2	011
3	010
4	110
5	111
6	101
7	100

b) Les codes autocorrecteurs

- le code p parmi n (ou code auto contrôlable)

Ces codes possèdent n éléments binaires dont p sont nécessairement à "1". La position de ces "1" permet de reconnaître un élément codé. Exemple le code 2 parmi 5 (ou code 0 1 2 4 7) utilisé dans les centraux téléphoniques pour la transmission d'informations numériques. Un chiffre de 0 à 9 est représenté par 2 éléments binaires à "1" parmi 5.

Décimal	2 parmi 5 7 4 2 1 0
1	0 0 0 1 1
2	0 0 1 0 1
3	0 0 1 1 0
4	0 1 0 0 1
5	0 1 0 1 0
6	0 1 1 0 0
7	1 0 0 0 1
8	1 0 0 1 0
9	1 0 1 0 0
0	1 1 0 0 0

Le nombre en décimal est donné par la somme des 2 poids dont les bits valent "1".

$$\text{Ex. } (1)_{10} = \text{poids } 1 + \text{poids } 0$$

$$(4)_{10} = \text{poids } 4 + \text{poids } 0$$

$$(9)_{10} = \text{poids } 7 + \text{poids } 2$$

$$(0)_{10} = \text{poids } 7 + \text{poids } 4 ; \text{ "0" est codé avec la combinaison restante}$$

Il est facile de contrôler puisqu'il suffit de vérifier que pour chaque combinaison il n'y a bien que 2 éléments binaires à l'état "1".

Ce système est très utilisé dans les systèmes électromécaniques.

- le code à contrôle de parité

C'est un code qui permet d'ajouter un élément binaire au caractère numérique ou alphanumérique déjà codé. Cet élément binaire supplémentaire est choisi de telle sorte que pour l'ensemble des bits (caractères + bit de parité) on ait un nombre pair ou impair de "1" ; celui-ci ne contient aucune information significative et sert uniquement au contrôle.

Exemple : Décimal code binaire code à contrôle de parité

5	0101	0101 0
7	0111	0111 1

← Bit de parité

2.3) Les codes alphanumériques

Ces codes représentent tous les caractères (chiffres, lettres et signe de ponctuation...) présents sur un clavier. Ils contiennent au moins 6 bits ($2^6 = 64$).

a) le code ASCII (American Standard Code for Information Interchange)

Il permet la représentation en binaire des chiffres, lettres, signes de ponctuation et commandes sur les claviers des machines à écrire. Il comporte 7 éléments binaires utiles et un élément binaire de parité. L'abondance des combinaisons possibles ($2^7 = 128$) a permis d'introduire les symboles de l'arithmétique, une différenciation des lettres majuscules et minuscules, des accents, des tabulations, des séparateurs...

éléments binaires de forts poids

éléments binaires de faibles poids

	(000)	(001)	(010)	(011)	(100)	(101)	(110)	(111)
	0	1	2	3	4	5	6	7
(0000) 0	NUL	DLE	SP	0	à	P	\	p
(0001) 1	SOH	DC1	!	1	A	Q	a	q
(0010) 2	STX	DC2	"	2	B	R	b	r
(0011) 3	ETX	DC3	#	3	C	S	c	s
(0100) 4	EOT	DC4	\$	4	D	T	d	t
(0101) 5	ENQ	NAK	%	5	E	U	e	u
(0110) 6	ACK	SYN	&	6	F	V	f	v
(0111) 7	BEL	ETB	'	7	G	W	g	w
(1000) 8	BS	CAN	(	8	H	X	h	x
(1001) 9	HT	EM	)	9	I	Y	i	y
(1010) A	LF	SUB	*	:	J	Z	j	z
(1011) B	VT	ESC	+	;	K	[	k	é
(1100) C	FF	FS	,	<	L	ç	l	ü
(1101) D	CR	GS	-	=	M	]	m	è
(1110) E	SO	RS	.	>	N	^	n	~
(1111) F	SI	US	/	?	O	_	o	DEL

Exemple : (2) a pour code (32)_H soit (0110010)₂

(A) = (41)_H = (1000001)₂ (a) = (61)_H = (1100001)₂

Signification des commandes rencontrées dans le tableau

NUL = rien	NAK = Réponse négative	VT = Tabulation verticale
SOH = début d'en-tête	SYN = Synchronisation	SP = Espace
STX = début de texte	ETB = Fin de transmission de bloc	FF = Page suivante
ETX = Fin de texte	CAN = Annule le caractère transmis	DEL = Suppression d'un caractère
EOT = Fin de transmission	EM = Fin physique du support	CR = Retour chariot
ENQ = Demande	SUB = Substitution	SO = Sortie de jeu standard
ACK = Accusé de réception	ESC = idem shift	SI = Retour au jeu standard
BEL = Sonnette	FS = Séparateur de fichiers	DLE = Change les caractères suivants
BS = recul d'un caractère	GS = Séparateur de groupes	DC1 = Commandes de périphériques
HT = Tabulation horizontale	US = Séparateur unitaire	DC2 = Commandes de périphériques
LF = Ligne suivante		DC3 = Commandes de périphériques
		RS = Séparateur d'enregistrements

b) Le code EBCDIC (Extended Binary Coded Decimal Interchange Code)

Il est utilisé comme code interne (représentation des caractères à l'intérieur du calculateur) ou comme code d'échange avec des organes d'entrée-sortie. Il comporte 8 bits soit $2^8 = 256$ combinaisons et n'utilise qu'une partie. Il est essentiellement utilisé sur les machines IBM.

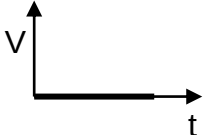
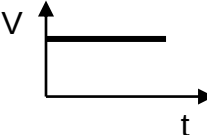
IV. LA LOGIQUE COMBINATOIRE

1. Définition

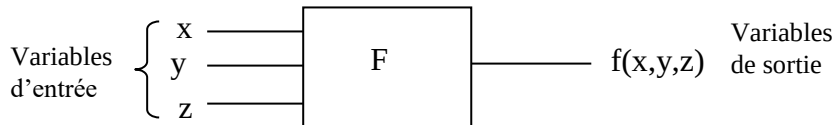
C'est une combinaison de variables logiques à l'entrée d'un système permettant d'obtenir une logique souhaitée à la sortie. L'état des sorties est une fonction déterminée par l'état des entrées.

Une variable binaire ou logique est une variable ne pouvant prendre que 2 états distincts et non simultanés qu'on notera "0" et "1". (Ex. Un interrupteur est fermé (état 1) ou ouvert (état 0)).

Elle peut représenter un contact, une lampe, un potentiel, ...

variable	Etats logiques	
	0	1
un contact	Contact ouvert 	Contact fermé 
une lampe	Lampe éteinte 	Lampe allumée 
un potentiel	$V = 0\text{ V}$ 	$V = 5\text{ V}$ 

Une fonction binaire ou logique est une fonction qui associe une image $f(x)$ à une ou plusieurs variables logiques d'entrée. L'état de la fonction $f(x)$ ne peut prendre que 2 états "0" ou "1".



$F(x,y,z) = 0$ ou 1 suivant l'état des entrées x, y, z . $F(x,y,z)$ est aussi appelé équation de la fonction F .

La table de vérité d'une fonction est un tableau qui, pour chaque combinaison possible des variables d'entrée, donne la valeur prise par la fonction. Une table de vérité définit complètement une fonction ou un opérateur logique.

3 variables d'entrée d'où $2^3 = 8$ combinaisons

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

3 variables de sortie

Un opérateur logique est un dispositif technologique permettant de réaliser une fonction logique. Cet opérateur est défini par une équation.

2. Fonctions et opérateurs logiques

a) Fonctions binaires d'une variable

Soit "a" une variable d'entrée. Examinons à l'aide d'une table de vérité toutes les fonctions qui peuvent se concevoir sur cette seule variable.

x	F_0	F_1	F_2	F_3
0	0	0	1	1
1	0	1	0	1

F_0 est une fonction qui vaut toujours 0 : c'est une constante.

F_1 est une fonction qui recopie l'état de "x" : c'est une fonction "OUI" ou identité. $F_1 = x$

F_2 est une fonction "inversion", également appelée fonction "NON".

F_2 est l'inverse de "x" ou le complément de "x" et est noté $F_2 = \overline{x}$ (lire x barre).

F_3 est une fonction qui vaut toujours 1 : c'est une constante

C'est la fonction inversion qui va nous intéresser. Elle est aussi dite fonction "NON" ; "NO". Elle se note par le symbole ($\bar{}$) nommé «barre». Elle prend l'état du complément de la variable d'entrée.

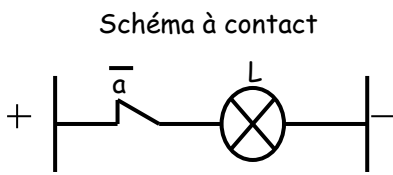


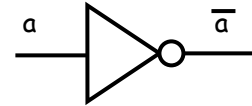
Table de vérité

a	L
0	1
1	0

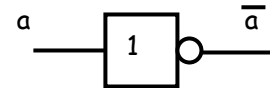
Équation

$$L = \bar{a}$$

Opérateur logique ou symbole



Norme américaine



Norme européenne

Le symbole de l'opérateur réalisant cette fonction est appelé inverseur (inverter).

C'est le petit cercle derrière ($\bigcirc$) qui symbolise la complémentation.

Il existe des circuits intégrés réalisant la fonction NON. Exemple référence : 7404 ; 74C04 ; 74HC04 ; 54HC04

b) Fonctions binaires de deux variables

- La fonction "OU" ; "OR" : addition logique notée (+) et se lit "ou"

C'est une fonction définie sur deux ou plusieurs variables d'entrée et qui vaut "0" si et seulement si toutes les variables d'entrée sont à "0".

Schéma à contact

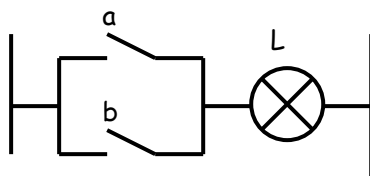


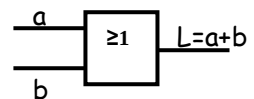
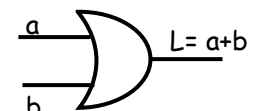
Table de vérité

a	b	L
0	0	
0	1	
1	0	
1	1	

Equation

$$L =$$

Opérateurs logiques



Exemple de circuit : 7432 ; 74HC32 ; 54HC32

- La fonction "ET" ; "AND" : produit logique notée (*) et se lit "et"

C'est une fonction définie sur deux ou plusieurs variables d'entrée et qui vaut "1" si et seulement si toutes les variables d'entrée sont à "1".

Schéma à contact

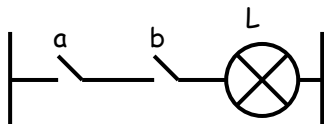


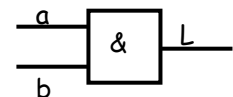
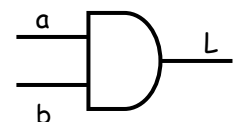
Table de vérité

a	b	L
0	0	
0	1	
1	0	
1	1	

Equation

$$L =$$

Opérateurs logiques



Exemple de circuit : 7408 ; 74HC08 ; 54HC08

3. Algèbre de BOOLE

a) DEFINITION

L'algèbre de Boole est un raisonnement logique dont les seules réponses possibles sont **oui** ou **non**. L'ensemble des opérations formelles appliquées à ces propositions forme une structure mathématique. Elle est applicable à l'étude des systèmes binaires, c'est à dire aux systèmes possédant 2 états s'excluant mutuellement.

Les états logiques sont représentés par les nombres 0 et 1.

b) Formules élémentaires de l'algèbre de Boole

Relative à l'opération somme logique (+)

$$x + 0 = x$$

$$x + 1 = 1$$

$$x + x = x$$

$$x + \overline{x} = 1$$

Relative à l'opération produit logique (*)

$$x * 0 = 0$$

$$x * 1 = x$$

$$x * x = x$$

$$x * \overline{x} = 0$$

c) Propriétés relatives aux opérateurs <<ET>> <<OU>> et <<NON>>

✓ élément neutre

 $x + 0 = x \rightarrow 0$ est l'élément neutre pour l'addition logique. $x * 1 = x \rightarrow 1$ est l'élément neutre pour la multiplication logique.

✓ commutativité

$$x + y = y + x \quad x * y = y * x$$

✓ associativité

$$(x + y) + z = x + (y + z) = x + y + z$$

$$(x * y) * z = x * (y * z) = x * y * z$$

✓ idempotence

Il n'y a pas d'exposant ou de coefficient

$$x + x + x + x + x + \dots = x$$

$$x * x * x * x * x * x * x * \dots = x$$

✓ complément

$$x = \overline{\overline{x}} \quad x + \overline{x} = 1 \quad x * \overline{x} = 0$$

✓ Double distributivité

$$x(y + z) = xy + xz$$

$$x + yz = (x + y)(x + z)$$

d) Théorèmes généraux

✓ Théorème d'absorption : $x + xy = x$ ✓ Théorème d'allègement : $x + \overline{x}y = x + y$; $\overline{x} + xy = \overline{x} + y$ ✓ Théorème de De Morgan : $\overline{x + y} = \overline{x} * \overline{y}$; $\overline{x * y} = \overline{x} + \overline{y}$

Le complément d'une fonction s'obtient en remplaçant les produits logiques par des sommes logiques et inversement, puis en remplaçant chaque variable par son complément.

Exemple : $F = x + y\overline{z} + m =$

4. Table de vérité et équations logiques

Si une fonction possède **n variable**, la table de vérité relative à cette fonction est un **tableau de (n+1) colonnes et de 2ⁿ lignes**. Chaque ligne est représentative d'une combinaison des variables parmi les 2ⁿ possibles (n colonnes). Une dernière colonne est réservée à la valeur de la fonction pour la combinaison des combinaisons correspondantes.

Exemple :

fonction OU de 2 variables a et b.

a	b	F
0	0	0
0	1	1
1	0	1
1	1	1

L'expression d'une fonction logique peut être une somme de produits ou un produit de somme

Exemple de la fonction Z :

Somme de produits :

Z =-----

Produit de somme ;

Z =-----

a	b	c	Z
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

5. Les opérateurs logiques complets

Un opérateur logique complet est un opérateur qui permet de réaliser toutes les autres fonctions combinatoires vues précédemment. Ceci est une conséquence du théorème de De Morgan.

Les expressions : $\overline{x + y} = \overline{x} * \overline{y}$ et $\overline{x * y} = \overline{x} + \overline{y}$ qui sont les complément du "OU" et du "ET", montrent que il est possible de passer du OU au ET, et réciproquement. Ces 2 opérateurs sont le "NON-OU" appelé "NOR" et le "NON-ET" appelé "NAND".

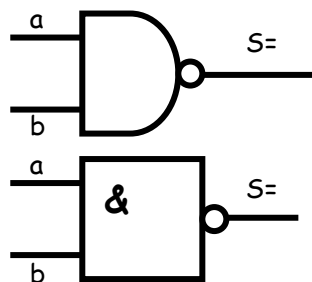
a) L'opérateur NAND (NON-ET)

"NAND" est la contraction de Not AND (Non-ET). En fait c'est un ET-NON, c'est-à-dire un ET suivi d'un inverseur. Il réalise une fonction ET complémentée.

Table de vérité

a	b	S
0	0	
0	1	
1	0	
1	1	

Symboles



Equation

S = -----

Exemple de circuit : 7400

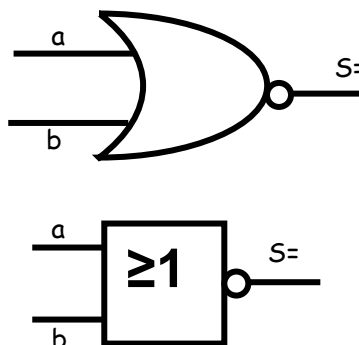
b) L'OPERATEUR NOR (NON-OU)

"NOR" est la contraction de Not OR (Non-OU). En fait c'est un OU-NON, c'est-à-dire un OU suivi d'un inverseur. Il réalise une fonction OU complémentée.

Table de vérité

a	b	S
0	0	
0	1	
1	0	
1	1	

Symboles



Equation

S = -----

Exemple de circuit : 7402

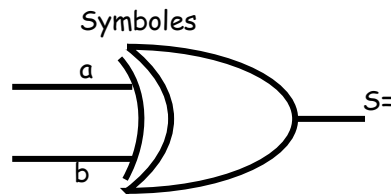
6. L'opérateur OU EXCLUSIF : XOR

La fonction OU EXCLUSIF ou Imparité est une fonction définie sur 2 variables et qui prend la valeur 1 si et seulement si une seule des variables est égale à 1.

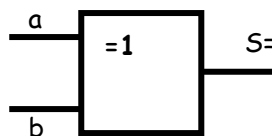
Son équation s'écrit : $S = a \oplus b$ (se lit "a OU EXCLUSIF b")

Table de vérité

a	b	S
0	0	
0	1	
1	0	
1	1	



Equation
S =



Exemple de circuit : 7486

Schéma à contact :

Le complément de la fonction OU-EX est la fonction coïncidence ou identité ou ET Inclusif:

Son équation s'écrit : $S = a \oplus \bar{b}$ (se lit "a OU EXCLUSIF barre b")

Faire sa table de vérité, son équation et son symbole.

7. Simplification des fonctions logiques

a) Méthode de l'algèbre de BOOLE

Les techniques de calcul pour simplifier une fonction de plusieurs variables ne sont pas systématiques. Néanmoins il sera bon de rechercher à mettre chaque variable sous la forme $x + \bar{x}$ ou $x \bar{x}$ pour éliminer la variable x .

Exemple : soit à simplifier la fonction : $F = \bar{a} b c + a b c + \bar{a} b \bar{c} + a b \bar{c}$

Autres exemples : exercices

Simplifier les fonctions (voir TD)

b) Méthode graphique : le tableau de KARNAUGH

C'est l'équivalent de la table de vérité. La méthode du tableau de Karnaugh va nous permettre d'effectuer des simplifications beaucoup plus rapides sans avoir à écrire de longues équations.

• Principe

C'est un tableau de 2^n cases, n étant le nombre de variables. Sur les lignes et colonnes, on place l'état des variables d'entrée codées en binaire réfléchi (code de Gray).

Dans chacune des cases on place l'état de la sortie pour les combinaisons d'entrées correspondantes.

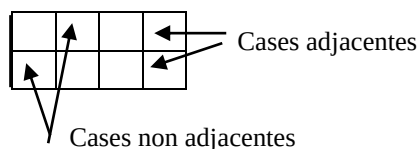
Dans l'exemple ci-contre, le nombre de variable est de 4 alors le tableau contiendra $2^4 = 16$ cases.

S	cd \ ab				
		00	01	11	10
cd	00				
	01				
	11				
	10				

• Méthode

La méthode consiste à mettre en évidence graphiquement tous les termes d'une fonction logique qui ne diffèrent que par l'état d'une seule variable (termes dits adjacents). Si une fonction logique dépend de 'n' variables d'entrée, on aura 2^n produits possibles (principe de la table de vérité).

Pour cela on réalise des groupements de cases adjacentes. Ces groupements de cases doivent être de taille maximale (nombre de cases max.) et égal à un multiple de 2^n . On cesse d'effectuer les groupements lorsque tous les "1" appartiennent au moins à un groupe.



4	2	2	4
3	1	1	3
3	1	1	3
4	2	2	4

Les cases portant le même numéro sont des exemples de cases adjacentes : regroupement possible

* règle de simplification

- on ne peut grouper qu'un nombre de cases correspondant à une puissance de 2 (2^k) : $2^0=1$; $2^1=2$; $2^2=4$; $2^3=8$;cases.

- le groupement de 2^k cases peut être en ligne ou en colonne ou en rectangle.

- il faut utiliser tous les "1" au moins une fois dans le groupement.

Il faut rechercher d'abord à regrouper le plus grand nombre de "1" dans les cases adjacentes : 2^3 ; 2^2 ; 2^1 ; 2^0 .

- une ou plusieurs cases peuvent être communes à plusieurs groupements.
- extraction de l'équation : un regroupement est un terme produit des variables qui s'ajoute aux autres regroupements. D'une case à l'autre si la variable change d'état, elle est éliminée dans le terme produit ; si elle ne change pas d'état et est à "1" on la prend tel qu'elle est et est à "0" on prend son complément.

Exemple : simplifier par TK

$$F1 = \overline{a} \overline{b} c + \overline{a} b c + a b c$$

$$F2 = \overline{a} b \overline{c} + a b \overline{c}$$

$$F3 = a \overline{b} \overline{c} + \overline{a} b c + a b c + a \overline{b} c$$

Simplification d'une fonction incomplètement définie à l'aide du TK

La fonction peut prendre une valeur x (indifférent 0 ou 1) pour certaines combinaisons des variables d'entrée. Dans la simplification par TK, nous utiliserons certaines cases x comme 1 si cela facilite le groupement des termes et 0 dans le cas contraire.

Exemple : utiliser le tableau de Karnaugh pour simplifier la fonction M

a	b	c	d	M
0	0	0	0	1
0	0	0	1	x
0	0	1	0	1
0	0	1	1	x
0	1	0	0	0
0	1	0	1	1
0	1	1	0	x
0	1	1	1	x
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	x

Si pratiquement certaines combinaisons des variables ne peuvent jamais exister, il est possible d'utiliser ces combinaisons pour simplifier la fonction.

Ces combinaisons «interdits» peuvent prendre la valeur 1 ou 0 dans le tableau de Karnaugh.

Exemple :

Soit un chiffre N du système décimal traduit en binaire à l'aide de 4 variables A, B, C, D . La sortie S du système à étudier prend la valeur 1 si $N \leq 5$.

TABLE DE VERITE

N	A	B	C	D	S
0	0	0	0	0	1
1	0	0	0	1	1
2	0	0	1	0	1
3	0	0	1	1	1
4	0	1	0	0	1
5	0	1	0	1	1
6	0	1	1	0	0
7	0	1	1	1	0
8	1	0	0	0	0
9	1	0	0	1	0
10 à 15	Cas impossibles				Non définie

TABLEAU DE KARNAUGH

$S =$ -----

8. Le logigramme

C'est une représentation graphique et symbolique d'une fonction simplifiée.

C'est un diagramme synoptique décrivant un problème logique, on l'appelle schéma logique. Il montre les différentes liaisons entre les opérateurs logiques pour un problème donné.

a) Définition d'un système logique complet

Une fonction logique quelconque d'un nombre quelconque de variables s'écrit avec les seuls opérateurs OU, ET et NON qui constituent un système logique complet ou groupe logique complet.

b) Les différents groupes logiques complets

On a très vite cherché à réduire le nombre d'opérateurs nécessaires à la constitution d'un groupe logique complet (ce qui simplifie évidemment la réalisation de circuits logiques). Il existe 4 groupes minimisés d'opérateurs.

- **Le groupe OU-NON pour obtenir le ET**

Faire le logigramme :

- **Le groupe ET-NON pour obtenir le OU**

Faire le logigramme :

*** L'opérateur NAND** : il permet à lui seul de réaliser les opérateurs NON, OU et ET

- Réalisation du NON :

- Réalisation du OU :

- Réalisation du ET :

*** L'opérateur NOR** : il permet à lui seul de réaliser les opérateurs NON, OU et ET

- Réalisation du NON :

- Réalisation du OU :

- Réalisation du ET :

d) Construction de logigrammes

Transformer les équations suivantes en porte NAND puis en porte NOR et faire les logigrammes :

$$S = a + \overline{b} c$$

$$K = x z + \overline{y} z$$

$$L = a \oplus b$$

V- Les circuits (intégrés) combinatoires

Objectifs :

Connaître et comprendre

- La définition d'un circuit combinatoire
- Ses diverses représentations et implémentations (multiplexeur, décodeur, ...)
- Les circuits combinatoires usuels, leur fonctionnement et leurs applications

Être capable de

- Analyser un circuit combinatoire existant
- Réaliser un nouveau circuit combinatoire à partir d'une spécification
- Transformer un circuit combinatoire en un circuit équivalent sous certaines contraintes.

1. Introduction

Circuit combinatoire : circuit dont les sorties dépendent uniquement de la combinaison des états des entrées à l'instant de l'observation.

La transmission de données nécessite fréquemment des opérations de conversion, de transposage et d'aiguillage. On utilise pour cela des circuits combinatoires. De plus au cœur de l'ordinateur se trouve un ensemble d'éléments combinatoires permettant d'effectuer les différentes opérations (arithmétiques et logiques).

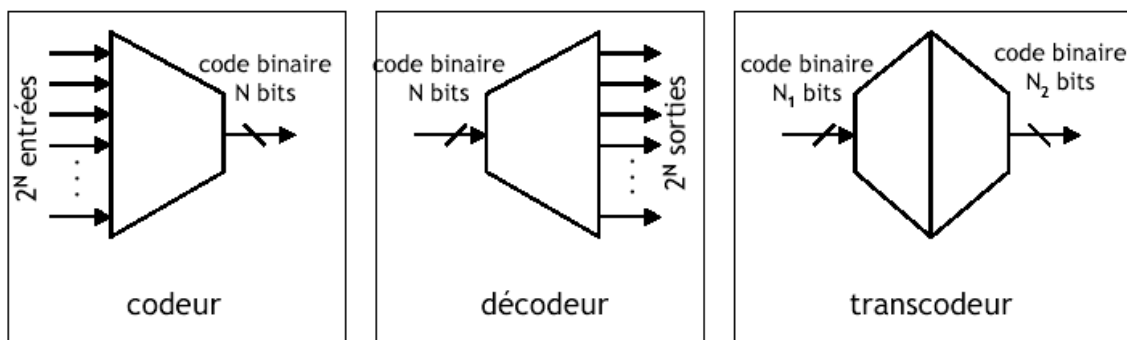
Voici une liste des principales fonctions standards combinatoires :

- L'encodeur de priorité
- Le décodage (X/Y) 1 parmi 2; 1 parmi 4; 1 parmi 8
- Le transcodage de nombres: BIN_BCD; BCD_BIN; BCD_7SEG; etc.
- Le multiplexage (MUX) 2 vers 1; 4 vers 1; 8 vers 1 et 16 vers 1
- Les opérations arithmétiques (addition, soustraction, ...)
- La comparaison (COMP) <, =, >
- ...

2. Circuits de transcodage

(Codeurs, décodeurs, convertisseurs ou transcodeurs)

Un circuit de transcodage (encore appelé transcodeur) transforme une information disponible en entrée sous forme donnée (généralement un code) en la même information, mais sous une autre forme (généralement un autre code). Il existe trois types de transcodeurs :

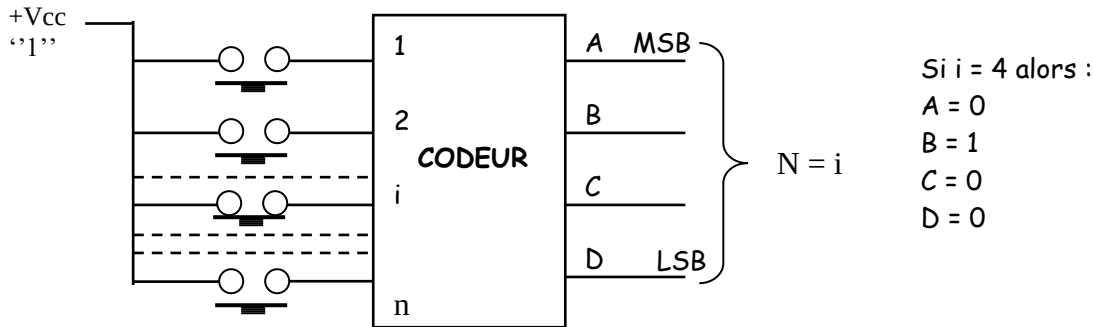


2.1 Les Codeurs

a) Définition

Un codeur est un dispositif qui traduit les valeurs d'une entrée dans un code choisi.

Par exemple un clavier comporte n touches. Chaque touche représentative d'un caractère est affectée d'un numéro. L'opération de codage consiste à donner à chaque numéro ou caractère un équivalent binaire c'est à dire un mot composé d'éléments binaires.



b) Intérêt du codage

Le codage des informations apporte une réduction du nombre de variables à traiter. Pour un clavier classique la quarantaine de touches se code facilement avec 6 variables.

2^n boutons se codent avec n variables.

c) Réalisation des codeurs

Dans sa version la plus simple un codeur est un ensemble de circuits 'OU'. Soit la table de codage pour les nombres de 1 à 9 en binaire :

N	A B C D
1	0 0 0 1
2	0 0 1 0
3	0 0 1 1
4	0 1 0 0
5	0 1 0 1
6	0 1 1 0
7	0 1 1 1
8	1 0 0 0
9	1 0 0 1

La sortie $A = 1$ si $N = 9$ ou $N = 8$

La sortie $B = 1$ si $N = 4$ ou $N = 5$ ou $N = 6$ ou $N = 7$

La sortie $C = 1$ si $N = 2$ ou $N = 3$ ou $N = 6$ ou $N = 7$

La sortie $D = 1$ si $N = 1$ ou $N = 3$ ou $N = 5$ ou $N = 7$ ou $N = 9$

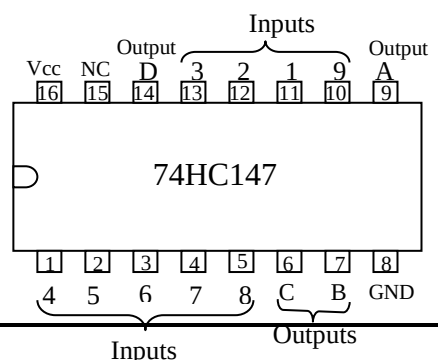
Réaliser le schéma de ce codeur :

Si aucune commande n'est envoyée au codeur les sorties A, B, C, D sont toutes nulles, ce qui exclut de coder le numéro 0.

Il existe des circuits intégrés codeurs.

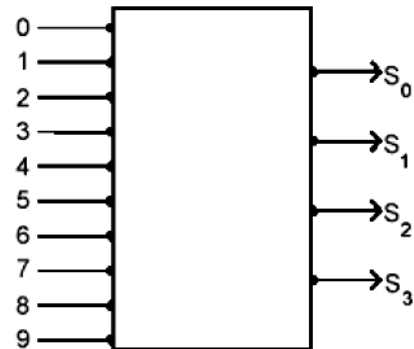
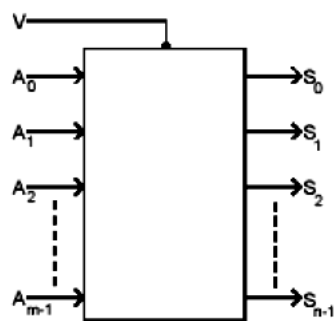
Exemple : 74HC147 ou 54HC147

10-to-4 line priority encoder : encodeur prioritaire 10 entrées décimales, 4 sorties binaires codeur prioritaire



Codeur (encodeur)

Circuit à $M=2^N$ entrées et N sorties qui code en binaire le rang de la seule entrée active.



Exemple: Codeur décimal-DCB 10 entrées, 4 sorties

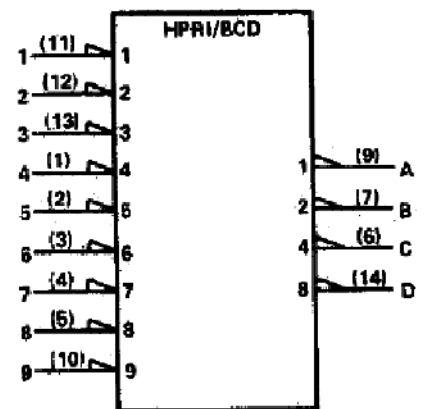
Table de vérité :

'147, 'LS147

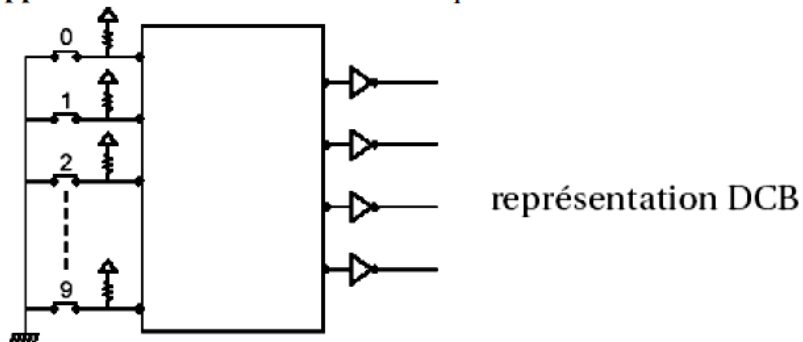
FUNCTION TABLE

INPUTS									OUTPUTS			
1	2	3	4	5	6	7	8	9	D	C	B	A
H	H	H	H	H	H	H	H	H	H	H	H	H
X	X	X	X	X	X	X	X	L	L	H	H	L
X	X	X	X	X	X	X	L	H	L	H	H	H
X	X	X	X	X	X	L	H	H	H	L	L	L
X	X	X	X	X	L	H	H	H	H	L	L	H
X	X	X	X	L	H	H	H	H	H	L	H	L
X	X	X	L	H	H	H	H	H	H	L	H	H
X	X	L	H	H	H	H	H	H	H	H	L	L
X	L	H	H	H	H	H	H	H	H	H	L	H
L	H	H	H	H	H	H	H	H	H	H	H	L

'147, 'LS147



Application : codeur de clavier numérique



Exemple de circuits intégrés codeurs

Circuit	Commentaire
74LS147	Encodeur de priorité décimal-BCD
74F148	codeur prioritaire à 3 bits

2.2 Les Décodeurs

a) Définition

Un décodeur est un circuit qui délivre une (ou des) information(s) lorsque la combinaison des variables binaires est représentative du (ou des) mot(s) - code choisi(s). Un décodeur réalise la fonction inverse d'un codeur.

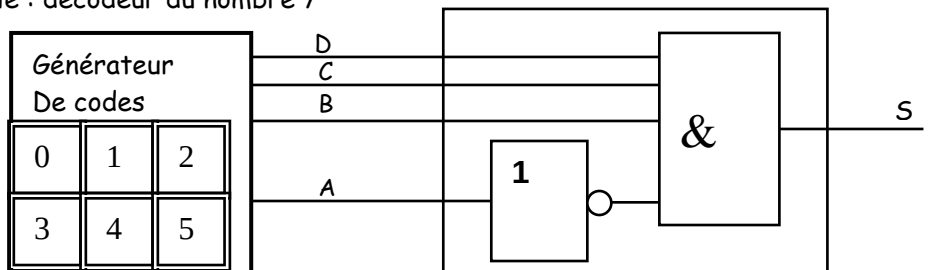
b) Réalisation des décodeurs

En pratique on trouve 2 types de décodage :

- **Le décodage d'une configuration (ou d'un mot-code) déterminée**

Le décodage est obtenu à l'aide d'un simple opérateur logique 'ET' vérifiant la simultanéité des variables binaires correspondantes à la configuration à décoder.

Exemple : décodeur du nombre 7



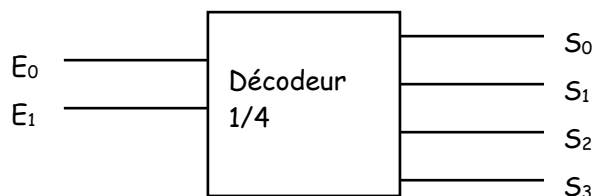
$$N = (ABCD)_2$$

$$S = 1 \text{ si } N = (0111)_2 = (7)_{10}$$

- **Le décodage de toutes les combinaisons d'un code**

Le circuit possède 2^n sorties dont une seule est active à la fois. La sortie active est déterminée par la combinaison des n entrées.

Exemple : décodeur 1 parmi 4 (Dresser la TdV, tirer les équations et réaliser le schéma logique)



Exercices

- Réaliser un décodeur 1 parmi 8.
- Réaliser un décodeur BCD / Décimal.

Exemple

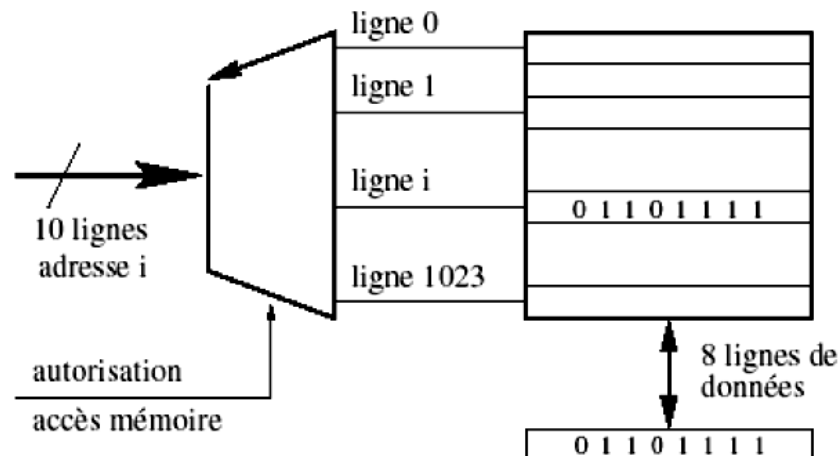
Décodeur DCB-décimal : 4 entrées, 10 sorties.

Remarque

La plupart des décodeurs sont dotés d'une ou plusieurs entrées de validation qui commandent son fonctionnement.

Applications

a. Adressage d'une mémoire



- une mémoire est un tableau d'éléments binaires (divisés en lignes et colonnes) ;
- pour lire un mot mémoire, il faut lui envoyer le numéro de ligne souhaité (adresse) ;
- souvent, le décodeur est interne à la mémoire.

Mise en cascade des décodeurs

Cela se fait par utilisation de l'entrée de validation.

Exemples :

- Réaliser un décodeur à 3 entrées en utilisant 2 décodeurs à 2 entrées.
- Réaliser un décodeur à 16 sorties à l'aide de décodeurs à 4 sorties.

Exemple de décodeurs CI

Circuit	Commentaire
74LS42	décodeur BDC - décimal
74LS138	Décodeur –démultiplexeur binaire –octal (3 vers 8)
74LS139	Double démultiplexeur-décodeur 2 vers 4, inverseur
74LS239	Double démultiplexeur-décodeur 2 vers 4

74LS156	Double démultiplexeur-décodeur 2 vers 4, inverseur, sorties C.O.
74LS238	Décodeur –démultiplexeur 3 vers 8
74LS154	Décodeur –démultiplexeur 4 vers 16, inverseur
74LS159	Décodeur –démultiplexeur 4 vers 16, inverseur, sorties C.O.

2.3 Les Transcodeurs (Convertisseurs)

Les transcodeurs sont des circuits qui permettent de passer d'un code à un autre.

Circuit à p entrées et k sorties qui convertit un nombre écrit dans un code C1 en un nombre écrit dans un code C2.

Exemple

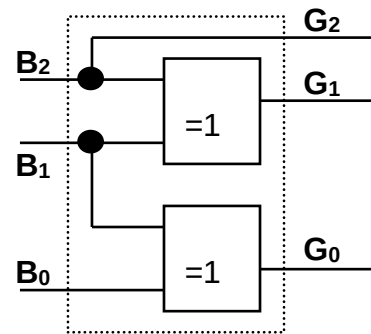
Code binaire → code Gray

Code DCB → code affichage chiffre (décodeur 7 segments)

Exemple 1 : transcodeur Binaire / Gray 3bits (tirer les équations et vérifier le schéma logique ci-dessous)

Table de vérité

B ₂	B ₁	B ₀	G ₂	G ₁	G ₀
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	1
0	1	1	0	1	0
1	0	0	1	1	0
1	0	1	1	1	1
1	1	0	1	0	1
1	1	1	1	0	0

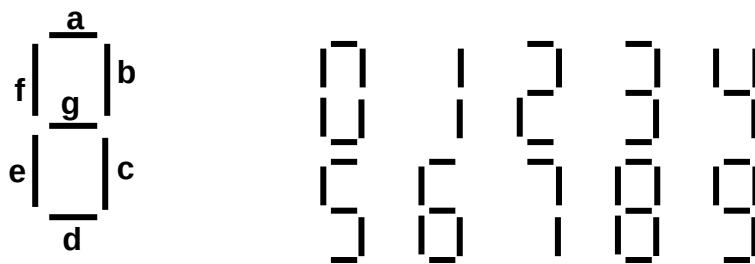


Exemple 2 : décodeur 7 segments

On appelle décodeur 7 segments ou transcodeur BCD / 7 segments le dispositif de transcodage permettant de passer du code BCD au code d'affichage de chiffres décimaux.

Soient a, b, c, d, e, f, g les variables correspondantes aux 7 segments. Si une variable est au niveau actif, le segment correspondant est allumé sinon il reste éteint.

Les chiffres étant formés de la façon suivante :



Exemple de synoptique :

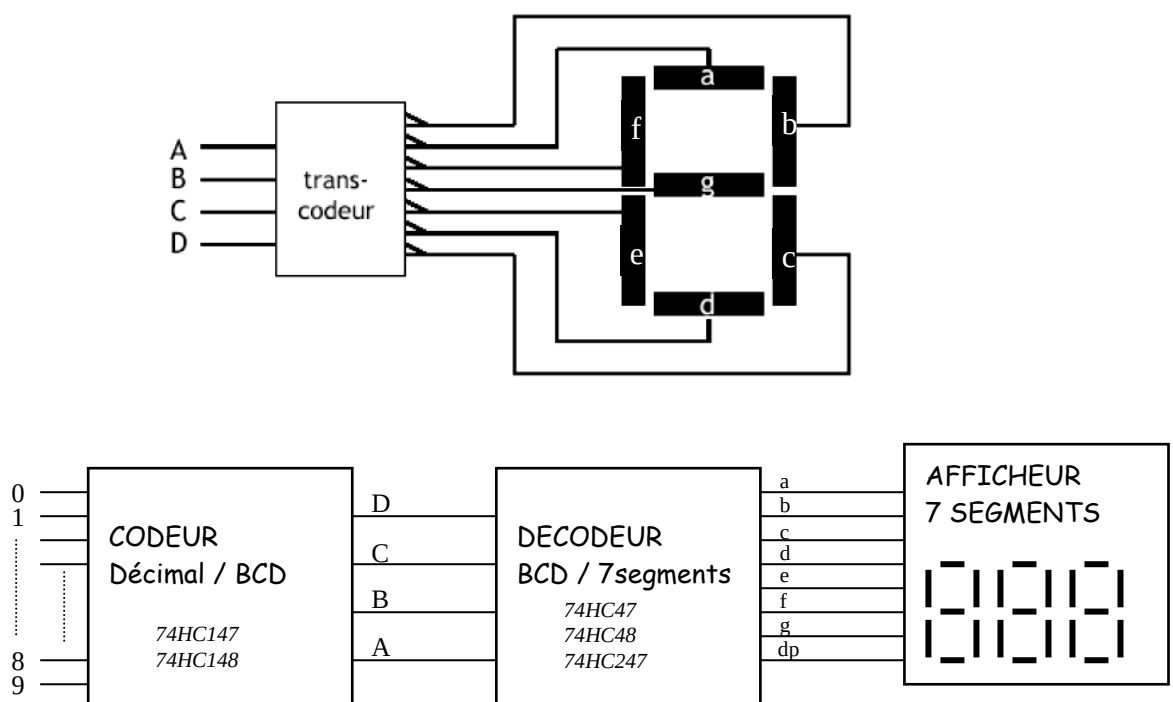


Table de vérité

(N) ₁₀	A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	0							
1	0	0	0	1							
2	0	0	1	0							
3	0	0	1	1							
4	0	1	0	0							
5	0	1	0	1							
6	0	1	1	0							
7	0	1	1	1							
8	1	0	0	0							
9	1	0	0	1							

Compléter la TdV et tirer les équations des 7 sorties.

Faire le schéma logique du décodeur

NB : Si le décodeur possède des sorties à collecteur ouvert, il est nécessaire d'adjoindre une charge résistive (R) à chaque sortie du décodeur. La valeur de R dépend de la valeur de l'intensité du courant débité en sortie ainsi que de la tension maximale autorisée sur les sorties du décodeur.

Quelques CI transcodeurs

Numéro du circuit	Commentaire ou fonction	Type d'afficheurs
7446	Décodeurs binaire/7segments	pour afficheurs à anode commune
7447	Décodeurs binaire/7segments	pour afficheurs à anode commune
74347	Décodeurs binaire/7segments	pour afficheurs à anode commune
74246	Décodeurs binaire/7segments	pour afficheurs à anode commune
74247	Décodeurs binaire/7segments	pour afficheurs à anode commune
74447	Décodeurs binaire/7segments	pour afficheurs à anode commune
7448	Décodeurs binaire/7segments	pour afficheurs à cathode commune
74248	Décodeurs binaire/7segments	pour afficheurs à cathode commune
74249	Décodeurs binaire/7segments	pour afficheurs à cathode commune

Il existe d'autres circuits décodeurs 7 segments 4511 et 4543 en CMOS.

3 LES MULTIPLEXEURS ET DEMULTIPLEXEURS

3.1 Les multiplexeurs (MUX)

a) Définition

Un multiplexeur est un circuit qui réalise un aiguillage de l'une de ses entrées vers une sortie unique d'information grâce à une commande C d'aiguillage appelé adresse.

Circuit à 2^n entrées d'informations, n entrées de sélection, et une sortie. Il permet l'aiguillage de l'une de ces entrées vers la sortie. La figure a) ci dessous présente la forme générale d'un MUX.

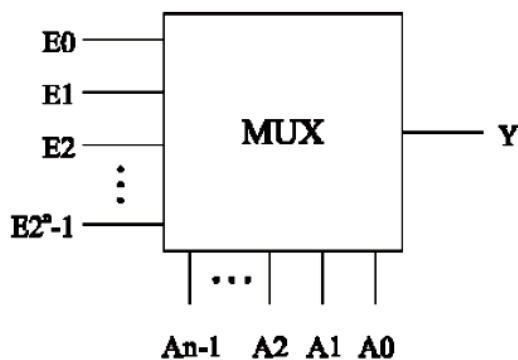


Figure a)

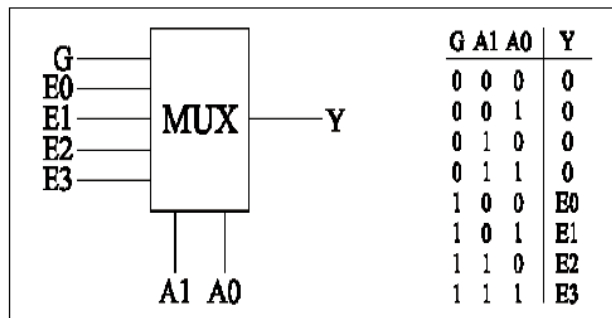


Figure b) : Caractéristiques du 74153.

La figure b montre le composant 74153 qui contient deux MUX 4 à 1. Le signal supplémentaire G (Strobe) est un signal d'activation du composant. Si G est inactif, la sortie du MUX sera obligatoirement inactive.

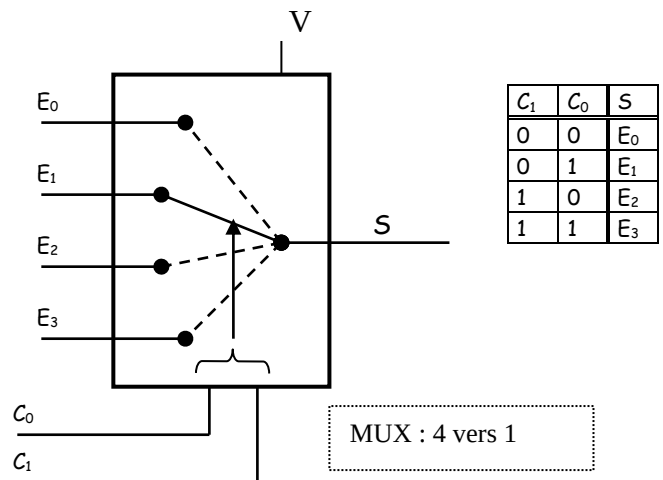
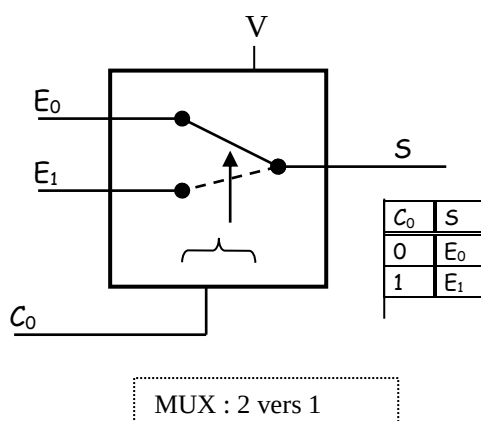
Il est nommé Multiplexeur 2^n vers 1.

Réaliser le schéma des multiplexeurs 2 vers 1 et 4 vers 1.

b) Multiplexeur 2 vers 1

Il possède 2 entrées de données (E_0, E_1), 1 commande C_0 .

La commande C_0 permet de sélectionner entre les 2 entrées (E_0 et E_1) : Multiplexeur 2 vers 1.



La position de l'interrupteur est fixée par une commande. Ainsi donc pour 2^n entrées il faut n éléments binaires de commande. V : Validation : c'est une entrée de validation (S: Strobe ou E: Enable) qui permet

de bloquer ou de libérer la sortie S. Exemple si $V = 0$: la sortie est bloquée ; si $V = 1$: la sortie est libérée.

La sortie peut être directe ou inversée ou même les deux cas de sorties.

L'inhibition de la sortie peut se traduire par une sortie "Haute impédance". Voir technologie Trois-états (Tri-state).

c) Multiplexeur 4 vers 1

Il possède 4 entrées (E_0, E_1, E_2 et E_3) et 2 commandes C_0 et C_1 qui permettent de sélectionner 1 entrée à envoyer à la sortie S : MUX 4 vers 1.

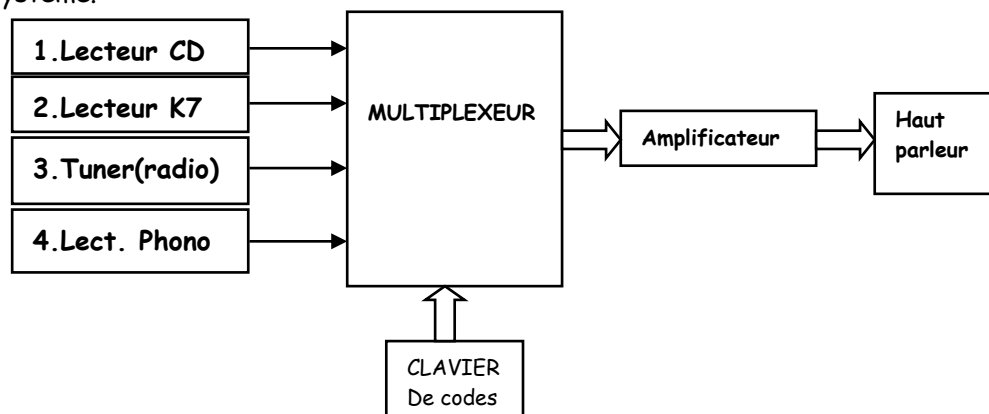
Chaque combinaison C_1C_0 est l'adresse d'une entrée à sélectionner.

TdV, équation, schéma logique.

d) Application

• Sélection d'informations

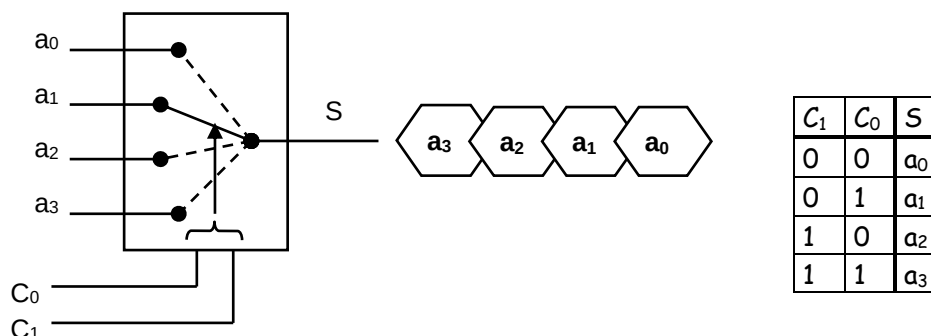
Exemple d'un système.



• Conversion parallèle - série

Soit un mot binaire $A = a_3 a_2 a_1 a_0$ disponible en mode parallèle c'est à dire sur 4 fils, chacun étant affecté à un élément binaire du mot.

Pour transmettre les éléments binaires en série, c'est à dire les uns après les autres sur un seul fil il faut d'abord transmettre a_0 puis a_1 puis a_2 puis a_3 . Ceci revient à sélectionner l'un des éléments binaires de A sur le fil unique de sortie série. Le multiplexeur est capable d'effectuer cette tâche si les combinaisons correspondantes sont placées successivement sur les commandes de sélection.



• Générateur de fonctions

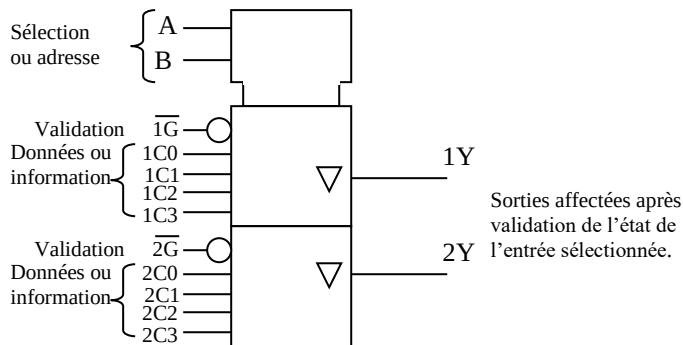
Les fonctions logiques peuvent être réalisées à partir des MUX. Les entrées de sélection (commande) sont alors les variables de la fonction. Voir TD

e) Circuit intégré multiplexeur

Exemples de CI

N° circuit	
74151	sélecteur-multiplexeur 8 vers 1
74253	Double sélecteur-multiplexeur 4 vers 1
74157 et 74158	Quadruple sélecteur-multiplexeur 2 vers 1
74257 et 74258	Quadruple sélecteur-multiplexeur 2 vers 1 avec sorties 3 états
4019	Quadruple sélecteur-multiplexeur 2 vers 1
4539	Double sélecteur-multiplexeur 4 vers 1

Double sélecteur-multiplexeur 4 vers 1 avec sorties 3 états : circuit 74LS253



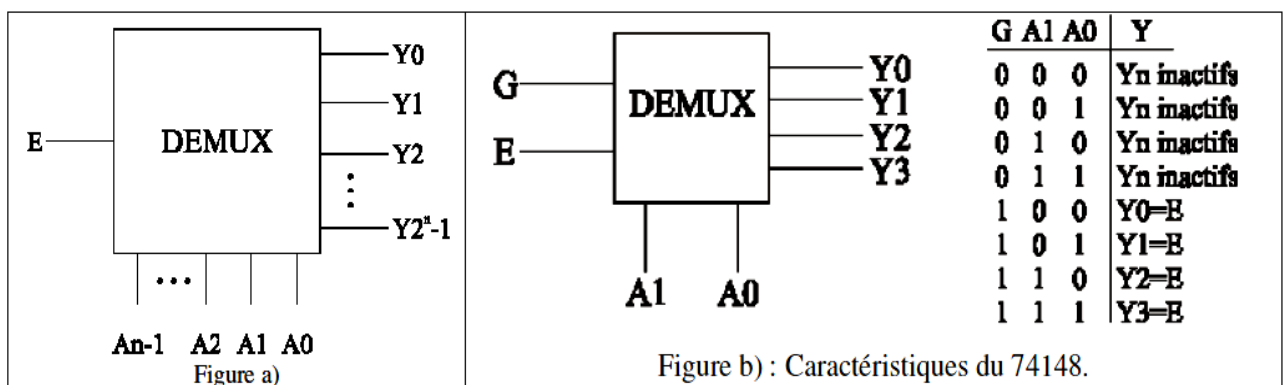
- * Deux entrées de sélection ou adresse : A et B qui permettent de sélectionner une entrée parmi les quatre.
- * Deux entrées de validation /1G et /2G qui permettent chacune de valider l'entrée sélectionnée sur la sortie.
- * Deux fois 4 entrées de données.
- * Deux sorties "3-états" séparées 1Y et 2Y.

3.2 Démultiplexeur (DEMUX)

Un Démultiplexeur est un dispositif réalisant également un aiguillage d'information. La différence entre le multiplexeur et le Démultiplexeur réside dans la circulation des informations.

Circuit à 2^n sorties, 1 entrée d'information, n entrées de commande. Il permet l'aiguillage d'informations de l'entrée vers l'une des sorties.

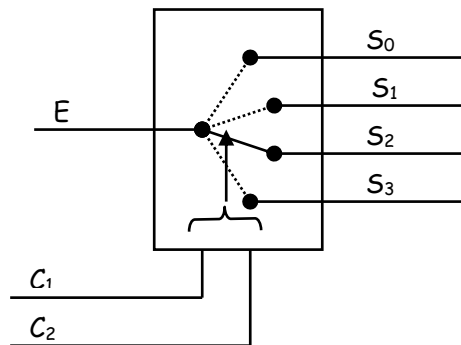
La figure a) ci-dessous présente la forme générale d'un DEMUX.



La figure b) montre un DEMUX 1 à 4. Le signal supplémentaire G (Strobe) est un signal d'activation du composant. Si G est inactif, toutes les sorties du DEMUX seront obligatoirement inactives.

Un Démultiplexeur distribue l'information d'entrée vers l'une des 2^n sorties ; la sélection de la sortie concernée étant effectuée par n variables de commande.

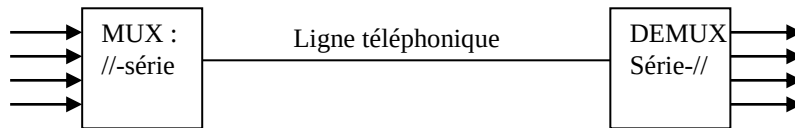
exemple : demultiplexeur 1 vers 4



C_1	C_0	S_3	S_2	S_1	S_0
0	0	0	0	0	E
0	1	0	0	E	0
1	0	0	E	0	0
1	1	E	0	0	0

Exemple circuit : 74LS155
Double Décodeur-Démultiplexeur 2 vers 4

Application : transmission de données entre ordinateurs par ligne téléphonique (ligne unique).



4 LES CIRCUITS INTEGRES ARITHMETIQUES

4.1 L'additionneur (Adder)

4.1.1 Le demi-additionneur (Half Adder)

C'est un circuit réalisant l'addition de 2 nombres binaires de 1 bit. Le résultat de l'opération comporte 2 parties : la somme Σ et la retenue r .

Soit l'addition de 'a' et 'b' de 1 bit :

Table de vérité

a	b	r	Σ
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

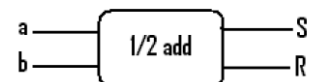
Equations

$$\Sigma = a \oplus b$$

$$r = ab$$

schéma

Schéma simplifié d'un demi-additionneur



4.1.2 L'additionneur complet (Full Adder)

Pour effectuer l'addition de 2 nombres binaires à plusieurs chiffres, la technique consiste à additionner successivement les chiffres de même poids avec éventuellement la retenue de l'addition précédente.

Soit l'addition de 'A' et 'B' de 4bits :

$$A = a_3a_2a_1a_0$$

$$B = b_3b_2b_1b_0$$

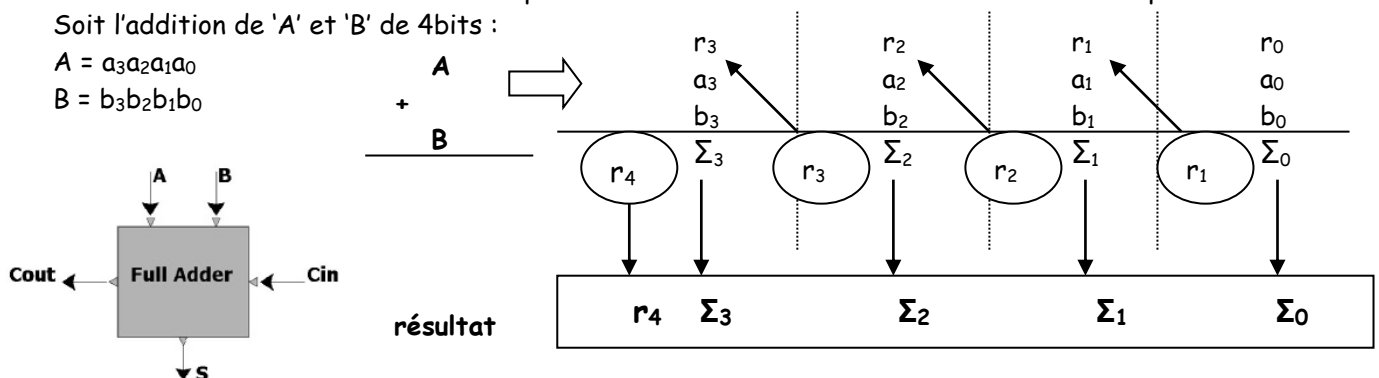


Table de vérité

a_n	b_n	r_n	Σ_n	r_{n+1}
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

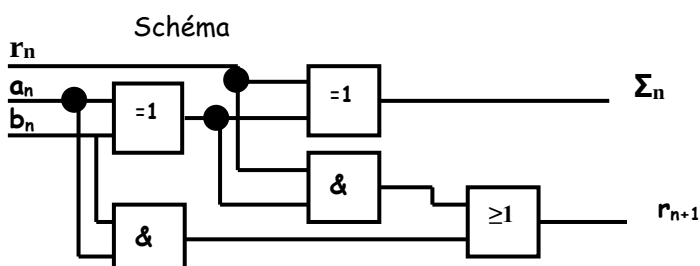
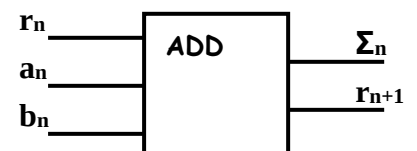
Equations

$$\Sigma_n = a_n \oplus b_n \oplus r_n$$

$$r_{n+1} = a_n b_n + (a_n \oplus b_n) r_n$$

L'additionneur ainsi réalisé est appelé additionneur complet.

Symbole

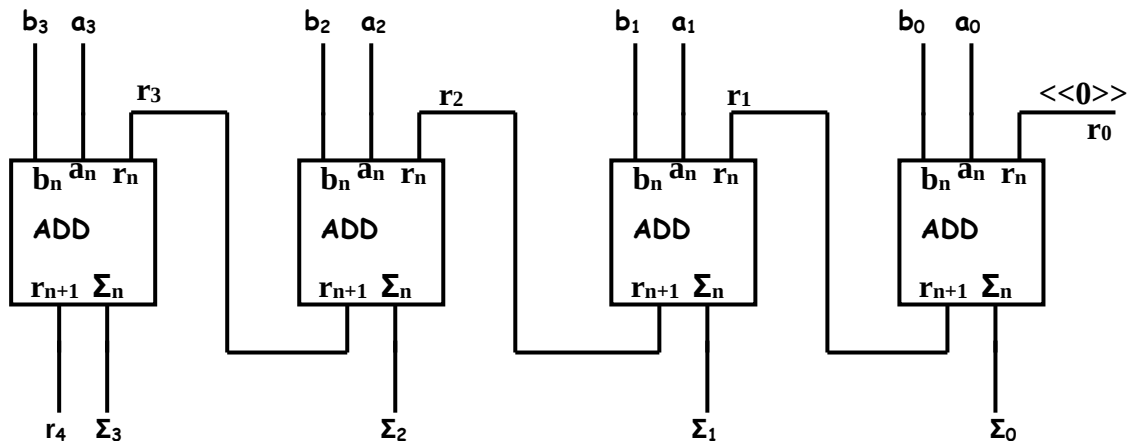


L'addition de 2 nombres de n bits nécessite n additionneurs complets, la retenue appliquée sur les plus faibles poids est nulle et chaque retenue calculée est appliquée au chiffre de poids immédiatement supérieur.

EXEMPLE : l'addition de 'A' et 'B' (Additionneur à propagation de retenue)

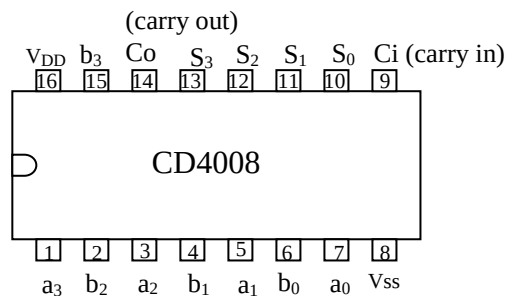
$$A = a_3a_2a_1a_0$$

$$B = b_3b_2b_1b_0$$



Exemple de circuit: 7483; CD4008 (4 bits Full Adder)

Carry : retenue



4.2 Le soustracteur (réaliser le circuit et réaliser un circuit additionneur-soustracteur)

4.3 Le comparateur

Un comparateur est un circuit capable de détecter l'égalité entre 2 nombres A et B et éventuellement d'indiquer le nombre le plus grand ou le plus petit.

4.3.1 Principe

Pour effectuer la comparaison de 2 nombres deux techniques sont utilisées :

- **La soustraction des 2 nombres** : Si le résultat de l'opération $A - B$ est positif, cela signifie que A est supérieur à B. Si le résultat est nul les 2 nombres sont égaux.
- **Une comparaison bit à bit** : C'est cette méthode qui est utilisée dans les circuits électroniques. La comparaison s'effectue poids par poids en commençant par le chiffre le plus significatif.

Les nombres A et B ayant le même format, le nombre A est supérieur à B si son élément binaire le plus significatif est supérieur à celui de B. Si ces deux éléments binaires sont égaux la supériorité ou l'infériorité ne peut être déterminée que par l'examen des bits de poids immédiatement inférieur et ainsi de suite.

L'examen des poids successifs s'arrête dès que l'un des éléments binaires est supérieur ou inférieur à l'autre. Les 2 nombres sont égaux si après avoir examiné tous les éléments binaires, il n'a pas détecté de supériorité ou d'infériorité.

4.3.2 Comparateur donnant l'égalité de 2 nombres de 1 bits

Table de vérité

a	b	E
0	0	
0	1	
1	0	
1	1	

équation

schéma

4.3.3 Comparateur donnant l'égalité de 2 nombres de n bits

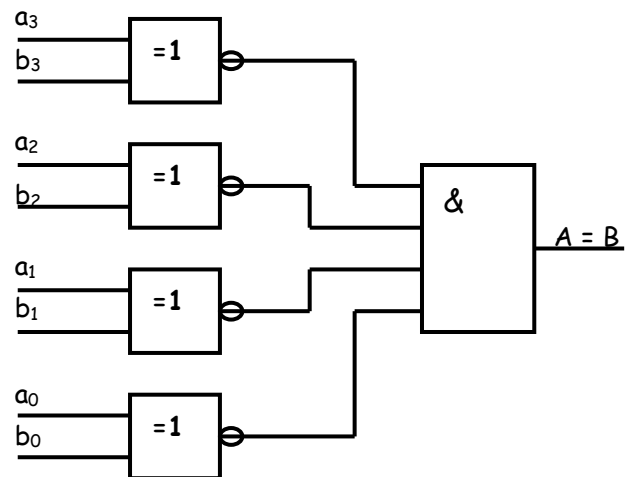
C'est le comparateur le plus simple. 2 nombres sont égaux si tous les chiffres de même poids sont égaux deux à deux.

Soit 2 nombres binaires A et B de 4 bits

$$A = a_3a_2a_1a_0 \quad B = b_3b_2b_1b_0$$

$$A = B \text{ si } (a_3 = b_3) \text{ ET } (a_2 = b_2) \text{ ET } (a_1 = b_1) \text{ ET } (a_0 = b_0)$$

Ce qui donne le schéma suivant :



4.3.4 Comparateur complet

La cellule élémentaire du comparateur comporte 3 entrées :

- les éléments binaires a_n et b_n de même poids ;
- une entrée A pour autoriser (valider) la comparaison

Compléter la table de vérité et tirer les équations :

Equations

$$S_n = \text{-----}$$

$$I_n = \text{-----}$$

$$E_n = \text{-----}$$

schéma

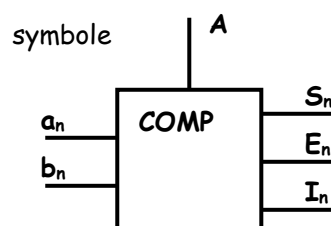


Table de vérité

A	a_n	b_n	$a_n = b_n$ E_n	$a_n > b_n$ S_n	$a_n < b_n$ I_n
0	0	0			
0	0	1			
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			

} Pas de comparaison

Comparateur de 2 bits

a_i	b_i	E_i	S_i	I_i
0	0	1	0	0
0	1	0	0	1
1	0	0	1	0
1	1	1	0	0

$$E_i = a_i = b_i = \overline{a \oplus b}$$

$$S_i = a_i > b_i = a \cdot \overline{b}$$

$$I_i = a_i < b_i = \overline{a} \cdot b$$

$$D_i = a_i \neq b_i = a \oplus b$$

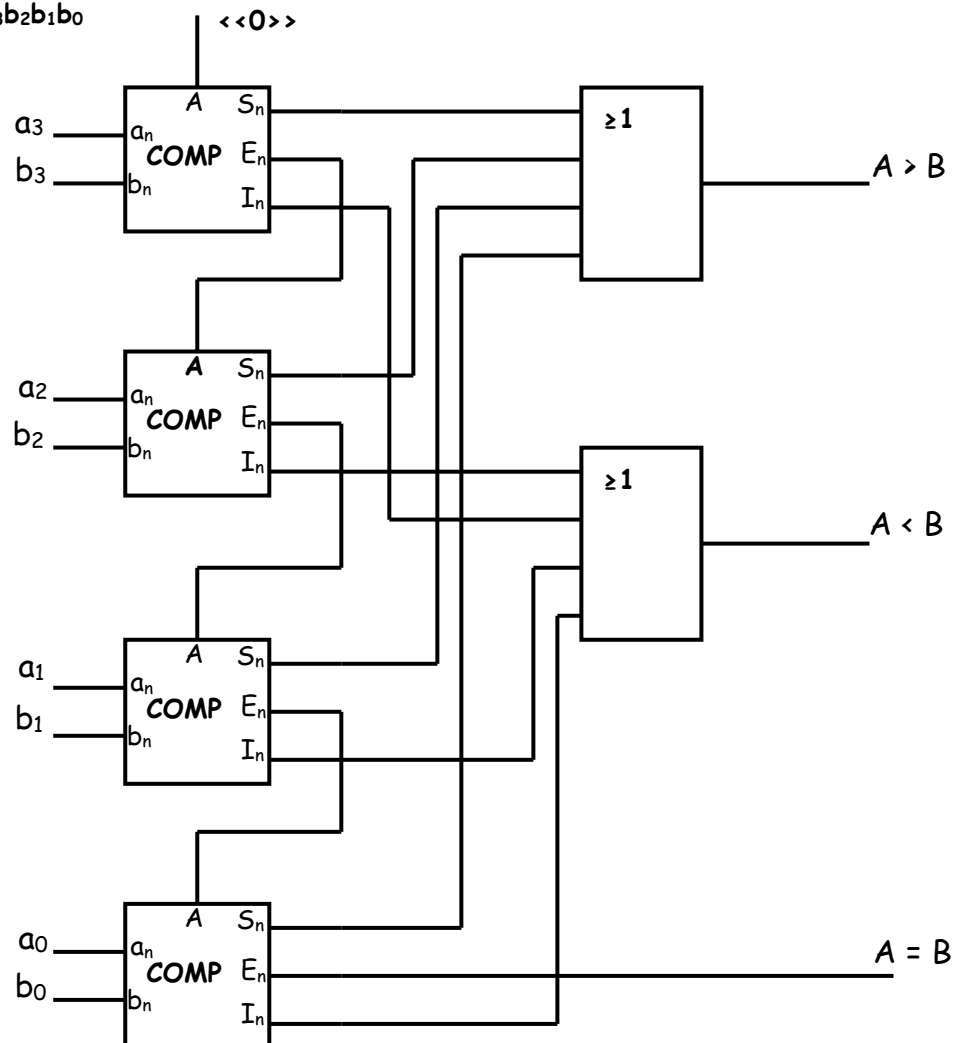
Pour étendre la capacité d'un comparateur, il faut procéder par mise en cascade.

Par analogie avec l'additionneur, la conception d'un comparateur pour des nombres de n bits se fait en cascade c'est à dire avec propagation des égalités partielles.

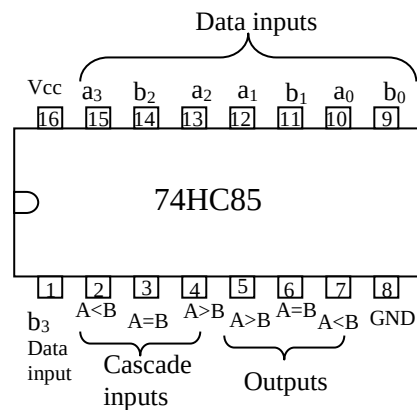
L'entrée d'autorisation est en fait la détection d'égalité des éléments binaires de poids supérieurs.

Exemple : Soit la comparaison de A et B

$A = a_3a_2a_1a_0$ $B = b_3b_2b_1b_0$



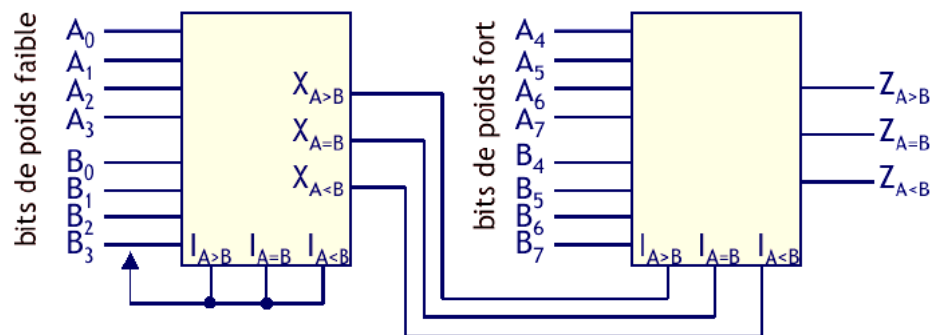
Exemple de circuit : 74HC85/ 54HC85 4 bits Magnitude Comparator



Cascade inputs : Entrées de mise en cascade lorsqu'on doit utiliser plus de deux circuits comparateurs pour comparer deux mots de plus de 4 bits chacun.

Outputs : sorties des cascades

Exemple du 74HC85 : comparateur 4 bits utilisé pour comparer des nombres de 8 bits.



4.3.5 L'Unité Arithmétique et Logique : UAL

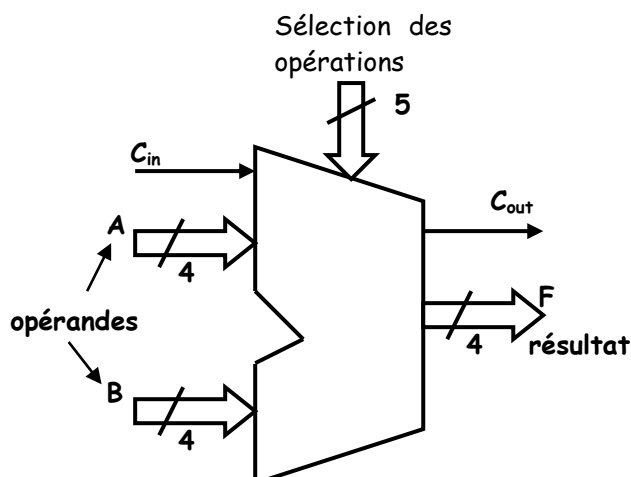
(Arithmetic Logic Unit : ALU)

Ce circuit est utilisé dans pratiquement tous les systèmes de calcul. C'est un opérateur capable d'effectuer un ensemble de traitement arithmétiques (addition, soustraction...) ou logiques (ET, OU...) sur des mots binaires de format donné.

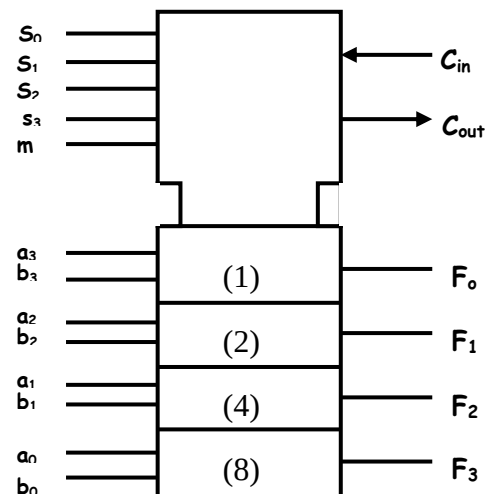
Le choix de l'opération est déterminé par des bits de commande. C'est donc un opérateur programmable.

Exemple de circuit d'Unité Logique et Arithmétique intégrée : Le **74 181**

Représentation fonctionnelle



Symbole logique



Sélection des opérations : se fait sur 5 bits ;
soit $2^5 = 32$ fonctions groupées en 16 opérations arithmétiques et 16 opérations logiques.

Application de l'ALU

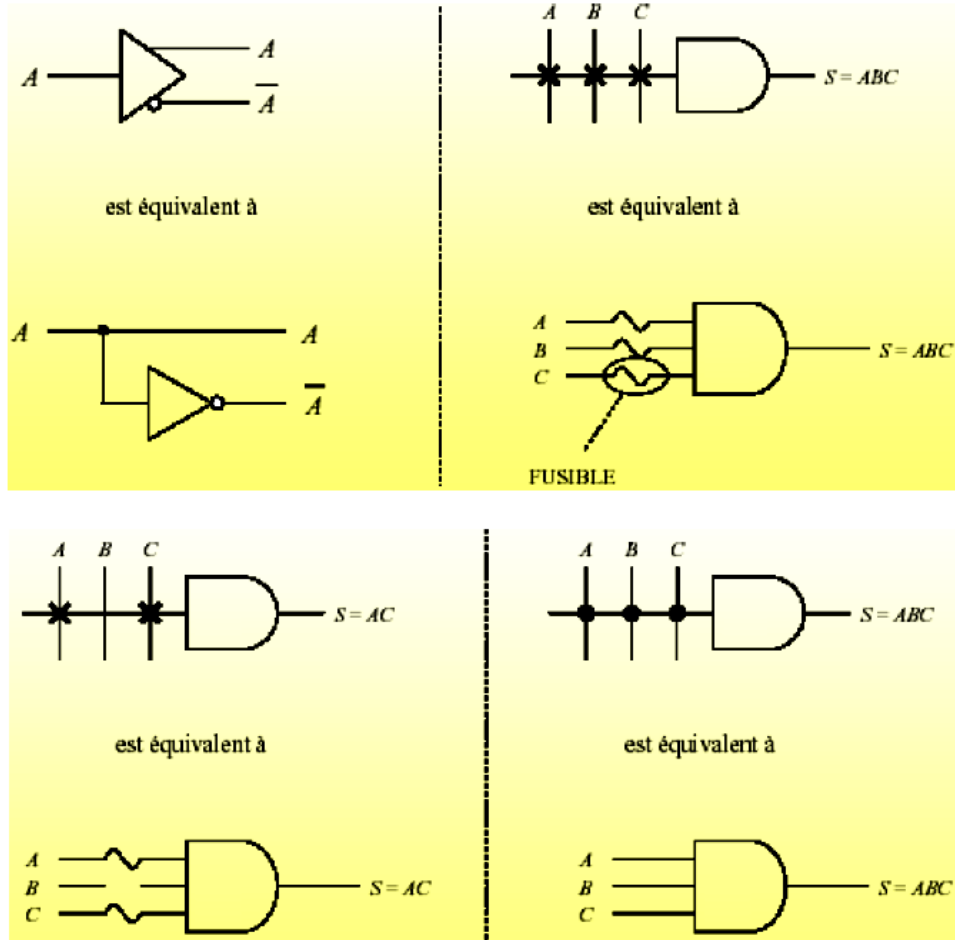
Selection $S_3 S_2 S_1 S_0$	m = 1 (Opérations logiques)	m = 0 (opérations arithmétiques)	
		$C_{in} = 0$ Pas de retenue entrante	$C_{in} = 1$ retenue entrante
0000	$F = \overline{A}$	$F = A$	$F = A \text{ plus } 1$
0001	$F = A + B$	$F = A + B$	$F = (A + B) \text{ plus } 1$
0010	$F = A \cdot B$	$F = A + B$	$F = (A + B) \text{ plus } 1$
0011	$F = 0$	$F = \text{moins } 1 \text{ (complément à 2)}$	$F = 0$
0100	$F = A \cdot B$	$F = A \text{ plus } A \cdot B$	$F = A + A \cdot B \text{ plus } 1$
0101	$F = B$	$F = (A + B) \text{ plus } A \cdot B$	$F = (A + B) \text{ plus } A \cdot B \text{ plus } 1$
0110	$F = A \oplus B$	$F = A \text{ moins } B \text{ moins } 1$	$F = A \text{ moins } B$
0111	$F = A \cdot B$	$F = A \cdot B - 1$	$F = A \cdot B$
1000	$F = \overline{A} + B$	$F = A \text{ plus } A \cdot B$	$F = A \text{ plus } A \cdot B \text{ plus } 1$
1001	$F = A \oplus B$	$F = A \text{ plus } B$	$F = A \text{ plus } B \text{ plus } 1$
1010	$F = B$	$F = A + \overline{B} \text{ plus } A \cdot B$	$F = (A + \overline{B}) \text{ plus } A \cdot B \text{ plus } 1$
1011	$F = A \cdot B$	$F = A \cdot B \text{ moins } 1$	$F = A \cdot B$
1100	$F = 1$	$F = A \text{ plus } A = 2A$	$F = A \text{ plus } A \text{ plus } 1$
1101	$F = A + B$	$F = (A + B) \text{ plus } A$	$F = (A + B) \text{ plus } A \text{ plus } 1$
1110	$F = A + B$	$F = (A + \overline{B}) \text{ plus } A$	$F = (A + \overline{B}) \text{ plus } A \text{ plus } 1$
1111	$F = A$	$F = A \text{ moins } 1$	$F = A$

NB : le signe + correspond au OU logique.

VI. les composants logiques programmables (voir Doc Sciences de l'ingénieur page 127)

I. Généralités

1. Notations



2. Principe des composants logiques

Toute fonction logique peut se mettre sous la forme de somme de produits ou de produit de sommes. Il semble donc naturel d'utiliser une structure comportant deux ensembles fonctionnels :

- ✓ un ensemble d'opérateurs ET organisé sous forme d'une matrice qui génère les produits de variables d'entrées, éventuellement complémentées.

- ✓ un ensemble d'opérateurs OU, appelée matrice OU, qui somme les produits.

L'idée est de ne conserver que les produits et les sommes qui nous intéressent. Pour cela, le composant programmable prévoit au départ toutes les connexions possibles et laisse à l'utilisateur le soin de couper (griller les fusibles) les liaisons qui ne l'intéressent pas. Voir figure 1 ci-dessous.

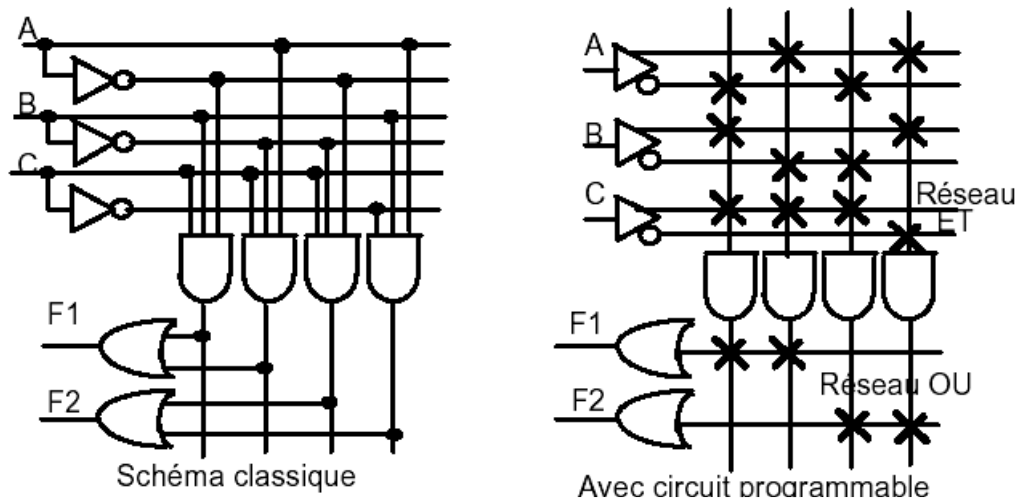
3. Classification des réseaux logiques combinatoires

Les réseaux logiques programmables recouvrent une grande gamme de produits qui diffèrent selon la partie programmable. La liste des différentes familles de composants programmables qui suit n'est pas exhaustive. Elle se veut une introduction aux différentes architectures de ces composants.

- ❖ PLE (Programmable Logic Element): Plan ET fixe, plan OU programmable.
- ❖ PAL (Programmable Array Logic): Plan ET programmable, plan OU fixe.
- ❖ PLA (Programmable Logic Array): Plan ET programmable, plan OU programmable.
- ❖ FPLA (Field PLA): PLA reprogrammable.
- ❖ PEEL (Programmable Electrically Erasable Logic)

- . EEPROM
- . EEPLA
- . EEPAL (GAL: Generic Array Logic)

Les différents plans sont illustrés par la figure 8 ci-dessous



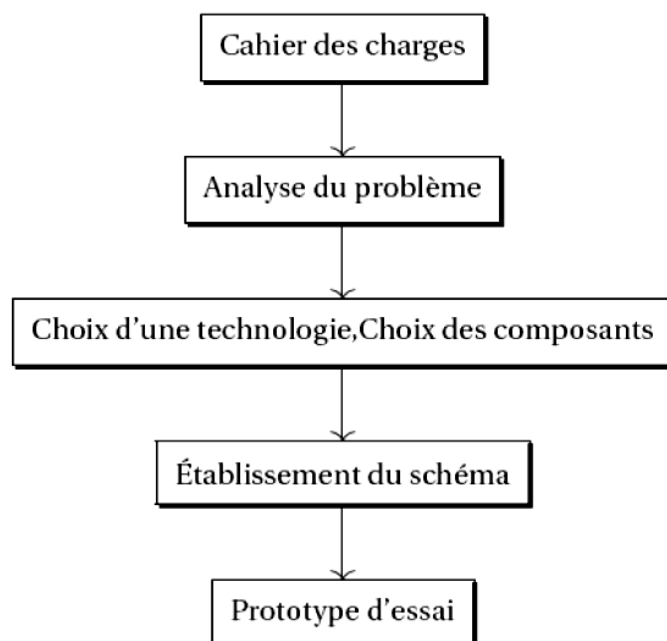
Intérêts de la logique programmable

Si un circuit est réalisé en logique programmable, il utilisera moins de composants puisque ceux-ci réalisent

directement les fonctions logiques désirées.

Un circuit ayant moins de composants sera habituellement moins coûteux à concevoir, réaliser et distribuer. La réduction du nombre de composants électronique tend aussi à augmenter la fiabilité des circuits et à réduire la consommation énergétique.

VII. Synthèse des circuits combinatoires



NB : Technologie des circuits logiques (voir doc Sciences de l'ingénieur page 117)

VII. TECHNOLOGIE DES CIRCUITS INTEGRES

1. Définitions

Composants discrets : Composants élémentaires contenus seuls dans un boîtier

Exemples : résistances, diodes, transistors

Puce (chip) : Plaquette de silicium sur laquelle est réalisé un ensemble de composants.

Circuit intégré : L'ensemble constitué par la puce et son boîtier

Exemples : opérateurs logiques, amplificateurs opérationnels

2. Les classes d'intégration

On distingue 4 classes d'intégration :

Intégration à petite échelle SSI (Single Size Integration) : environ 100 composants par cm^2 .

Intégration à moyenne échelle MSI (Medium Size Integration) : environ 1000 composants par cm^2 .

Intégration à grande échelle LSI (Large Size Integration) : environ 10000 à 100000 composants par cm^2 .

Intégration à très grande échelle VLSI (Very Large Size Integration) : environ $0,1$ à 10^6 composants par cm^2 .

3. Les familles de circuits intégrés

Il existe 2 familles de circuits intégrés :

- Les circuits intégrés **analogiques** ou **linéaires** (les amplificateurs opérationnels....)
- Les circuits intégrés **logiques** ou **numériques** (opérateurs logiques, bascules, compteurs, mémoires...)

a. Les circuits logiques

Il existe plusieurs familles de circuits technologiques. Les plus utilisées sont :

- La famille **TTL** (Transistor Transistor logic) à transistors bipolaires.
- La famille **CMOS** (Complementary Metal Oxyd Semi-conductor) à transistors à effet de champ.

i. La famille TTL

Il existe plusieurs sous-familles ou séries, de la plus ancienne à la plus récente.

Les principales différences entre elles sont la puissance moyenne consommée (P) et la rapidité de conduction (Tpd)

DESIGNATION

Type - Famille - Fonction

Type : 54 : Militaire ($T^\circ = -55^\circ\text{C}$ à $+125^\circ\text{C}$)

74 : Civil ($T^\circ = 0$ à $+70^\circ\text{C}$)

Séries : H-N -L -S -F-LS- AS- ALS

H: High speed, **L**: Low power, **S**: Schottky, **LS**: Low power Schottky

AS: Advanced Schottky, **ALS**: Advanced Low power Schottky, **F**: Fast

Aucune lettre = standard

Fonction : 2 ou 3 chiffres

Exemple : 74 LS 00 : Opérateurs NAND à 2 entrées

FAMILLES	P(mW)	Tpd(nS)
N	10	10
H	22	6
L	1	33
S	19	3
F	6	12
LS	2	9,5
AS	20	1,5
ALS	1	4

Les circuits TTL sont alimentés en +5V +/-5%

ii. La famille CMOS

Les circuits CMOS, récentes par rapport aux circuits TTL, occupent déjà la plus grande part du marché des circuits intégrés logiques. Elles se divisent en séries **HC-AC-CD4000**.

SERIES	Tpd(mW)	VDD(V)	P(mW)
HC	9	3-6	négligeable
AC	3	3-6	
CD4000	1	5-18	

DESIGNATION

a- Type - Famille - Fonction

Type : 54 : Militaire

74 : Civil

Famille : HC - AC - C

Fonction : 2 ou 3 chiffres

b- CD 4xxx (xxx = 000-999)

Exemples : 54 HC 10 / 74 HC 10 = NAND à 3 entrées

CD 4023 = NAND à 3 entrées

Les circuits CMOS sont alimentés entre +3V à +18V

b. Niveaux de fonctionnement

On appelle niveau de fonctionnement ou logique la plage de tension pour laquelle le circuit passe au niveau logique "0" ou "1". Ce niveau est différent pour les différentes familles de circuits logiques.

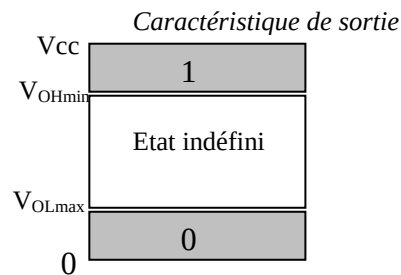
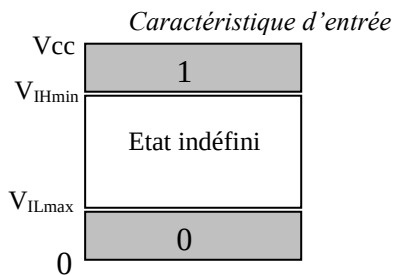
On définit pour les valeurs de tension en entrée (input) et en sortie (output) par :

* V_{IHmin} : Tension minimale en entrée (Input High Voltage) qui assure le niveau logique haut.

* V_{ILmax} : Tension maximale en entrée (Input Low Voltage) qui assure le niveau logique bas.

* V_{OHmin} : Tension minimale de sortie (Output High Voltage) qui assure le niveau logique haut.

* V_{OLmax} : Tension maximale de sortie (Output Low Voltage) qui assure le niveau logique bas.

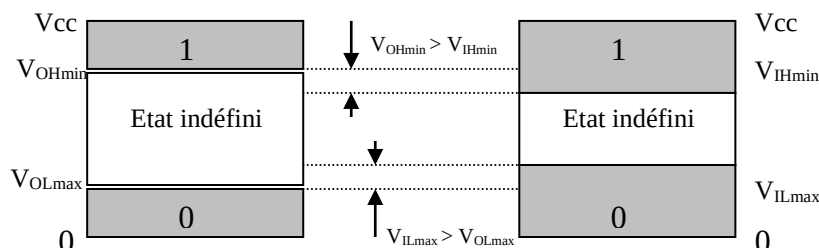


c. Compatibilité des circuits logiques

La compatibilité des circuits logiques réside dans le fait que si deux ou plusieurs circuits de technologies différentes doivent être reliés ensemble de respecter les niveaux logiques de chacun. Ainsi pour relier 2 circuits il faut respecter :

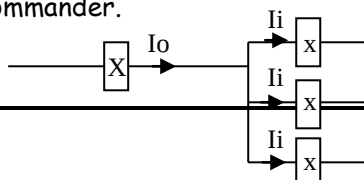
La compatibilité au niveau haut : $V_{OHmin} > V_{IHmin}$

La compatibilité au niveau bas : $V_{ILmax} > V_{OLmax}$



d. Facteur de charge : sortance

On appelle sortance N d'un circuit le nombre d'entrée N de portes logiques que la sortie de ce circuit peut commander.



Sortance N (à l'état haut) : I_{OH} / I_{IH}

Sortance N (à l'état bas) : I_{OL} / I_{IL}

le document constructeur fournit les données suivantes : I_{OH} , I_{IH} , I_{OL} , I_{IL}

Conclusion

Pour réaliser les fonctions logiques précédentes on utilise des circuits intégrés : **PORTES**. On distingue deux grandes familles de circuits logiques :

- Circuits **TTL** "Transistor Transistor Logic"
- Circuits **CMOS** "Complémentary Metal-Oxyde S.C"

Quelques propriétés (voir TP)

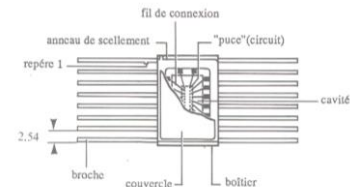
- *alimentation* : TTL: $V_{CC} = 5V$; CMOS: $V_{DD} = 3$ à $18V$
- *niveau bas d'entrée* TTL: 0 à 0,8 V; CMOS: 0 à 40% V_{DD}
- *niveau haut d'entrée* TTL: 2 à 5 V; CMOS: 60% à 100% V_{DD}
- *niveau bas de sortie* TTL: 0 à 0,4 V; CMOS: 0 à 10% V_{DD}
- *niveau haut de sortie* TTL: 2,4 à 5 V; CMOS: 90% à 100% V_{DD}

Les circuits de technologie TTL sont plus "rapides" que les circuits CMOS, par contre les circuits CMOS ont une "consommation" beaucoup plus faibles que les circuits TTL.

e. Les boîtiers des circuits intégrés

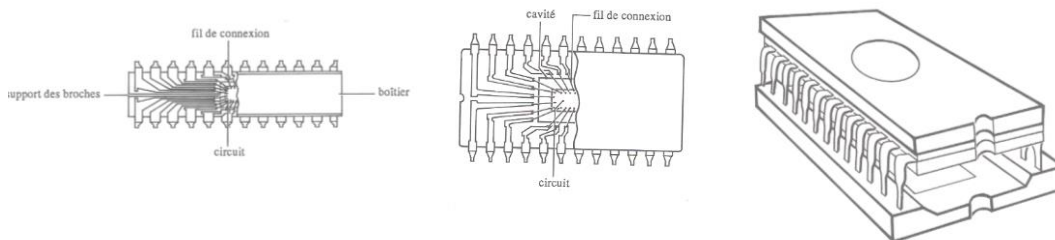
Les boîtiers ronds habituellement utilisés pour les circuits analogiques ont disparu dès que le nombre de connexions fut supérieur à quelques unités.

Les boîtiers plats (flat package) sont utilisés chaque fois qu'il existe un problème d'interconnexion ou de poids comme le matériel embarqué. Afin de limiter la taille des boîtiers les connexions sont réparties Autour du boîtier.

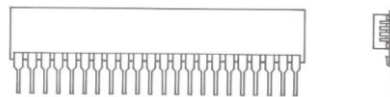


Les boîtiers DIL (dual in line) sont les boîtiers les plus fréquemment rencontrés. Ils ont 8 à 64 connexions réparties en 2 lignes. Le cadre peut être en plastique (Les boîtiers DIL plastique) ou en céramique (Les boîtiers DIL céramique). Les boîtiers DIL en céramique peuvent être équipés d'un système de refroidissement.

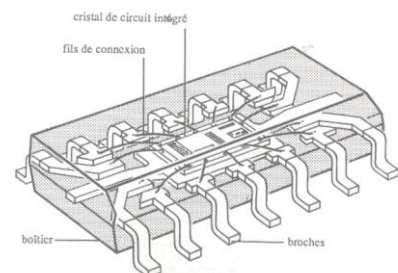
Ils sont robustes et peuvent être soit soudés soit montés sur des supports pour faciliter l'échange lors d'une panne.



Les boîtiers SIL (single in line) ont les connexions placées que d'un seul coté et sont souvent rencontrés pour les circuits dits grand public ou pour les circuits mémoires.



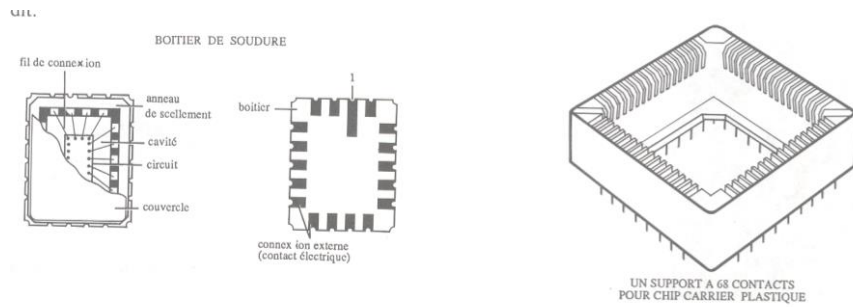
Les boîtiers SO (Small out line) sont des boîtiers semblables aux boîtiers DIL mais les connexions sont coudées afin de permettre une soudure du coté composant. Leur utilisation procure l'avantage de réaliser des circuits imprimés sans trous et permet une densité plus grande, donc une miniaturisation des équipements.



VUE EN COUPE D'UN BOÎTIER SO 14

Les boîtiers chip carrier sont des boîtiers enfichables dans des supports spéciaux. Ils n'ont pas de broches et liés électriquement à l'extérieur par des métallisations locales. Le support de ce type de boîtier dispose de lamelle métallique élastique assurant un

contacte de bonne qualité. Le support est soudé sur le circuit soit du coté composant soit du coté cuivre. Comme pour les boîtiers DIL, Les boîtiers chip carrier peuvent être équipés d'un système de refroidissement.



Les boîtiers pin grid array (réseau de connexions) sont utilisés pour les circuits ayant des connexions en nombre très important. Ils peuvent être soudés directement sur le circuit imprimé ou montés sur support. Les connexions sont réparties sur la périphérie du boîtier suivant une ou plusieurs tours. Ces boîtiers peuvent également être équipés d'un système de refroidissement.